香港中文大學統計學系
Department of Statistics
THE CHINESE UNIVERSITY OF HONG KONG

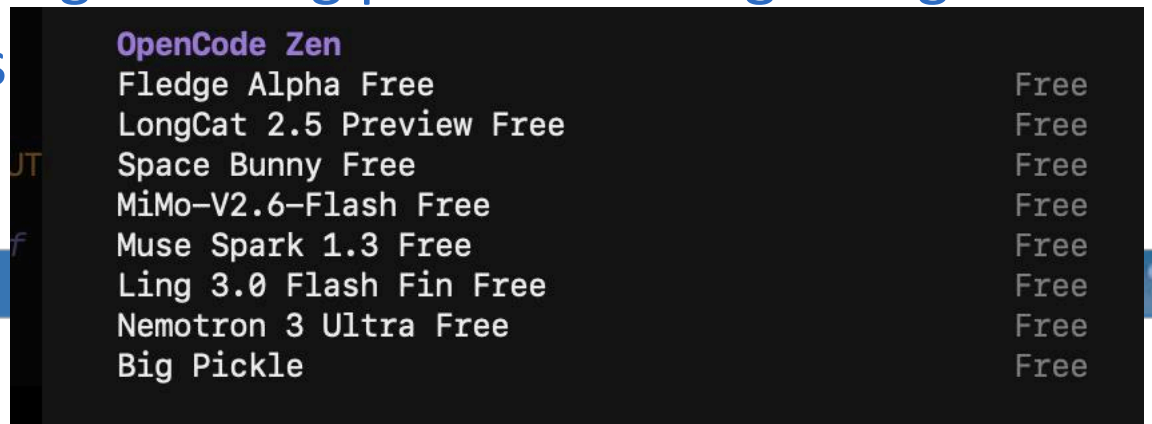
STAT6207
Applied Deep Learning

L6, 16 Oct 2026
By Dr. Ming Cheung



Announcement

- Midterm exam will be on 23 Oct 26 at room: CYT201, 202, 209B
- There will be two parts
  - Written: MC and short question
  - Programming: You can use your computer and agent coder
- No communication is allowed, and you are not allowed to use your phone.
- You can bring a cheat sheet (A4, double page)
- Make sure that you can run all demo and assignments on your computer
- Duration 2.5 hours, you will need to complete the written part first
- You will know the questions in Programming part at the beginning
- You will need to use free models



Lecture Room - next lecture (L7): CYT201, 202, 209B

Lecture	Week	Lecture Rooms
L6	16-Oct	BMS G18
L7 (Mid Term)	23-Oct	CYT201, 202, 209B
L8	30-Oct	CYT LT1A
L9	6-Nov	UCC C1
L10	13-Nov	CYT LT1A
L11	20-Nov	UCC C1
L12	27-Nov	UCC C1
L13	4-Dec	UCC C1
L14	11-Dec	BMS G18



Last Lecture

Some problems that we can solve…

- What can computer "see"

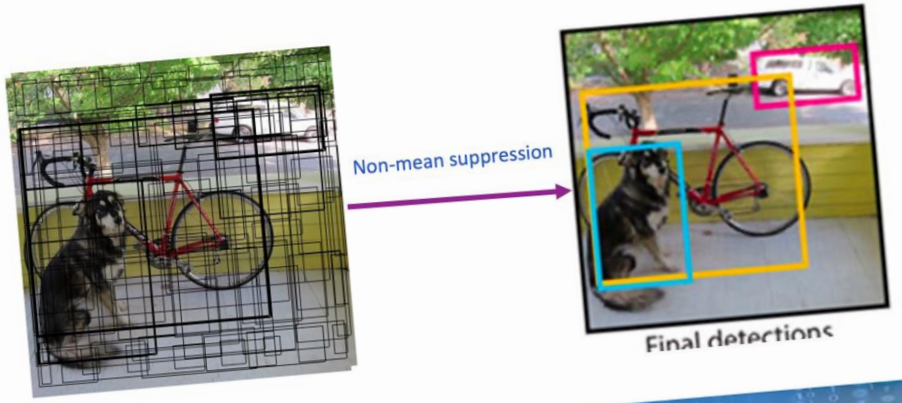


Nano Banana: changing perspective



28

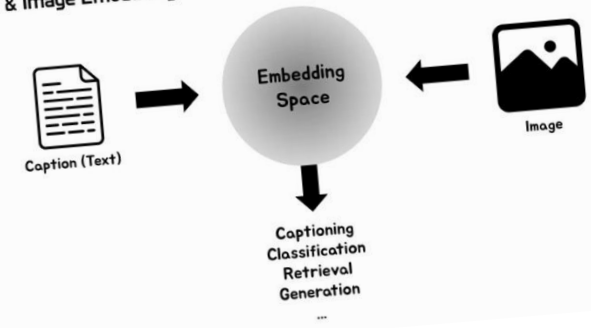
Yolo



Contrastive Language-Image Pre-Training (CLIP)

- During training
  - batches of (x, c) pairs are sampled from a large dataset, and the model optimizes a contrastive cross-entropy loss
  - encourages a high dot-product f(x) · g(c), for a pair x and c

◆ Text & Image Embedding



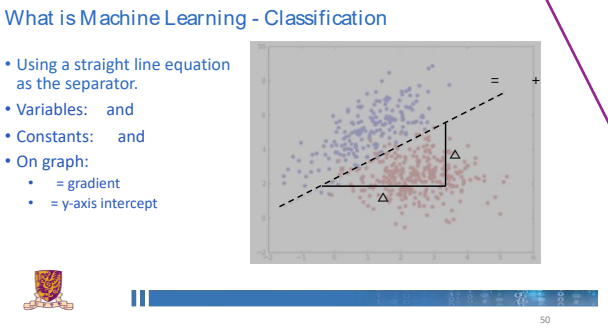
80

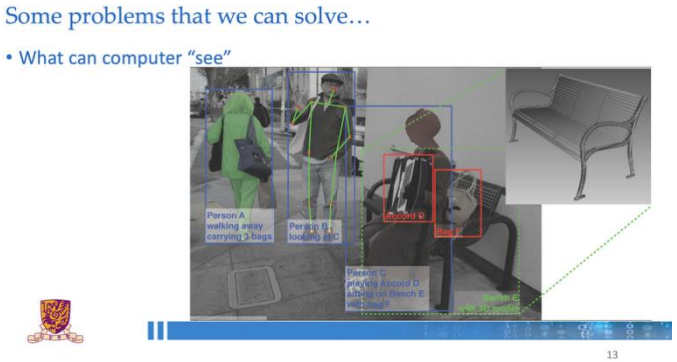
Course so far

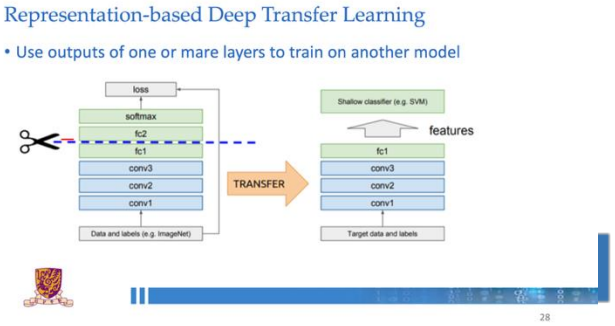
- How do we process images?

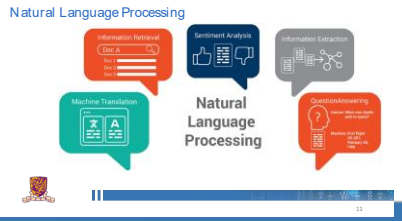
Machine Learning → Transfer Learning → Computer Vision → Structures?

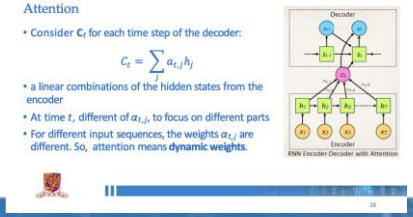
Transfer Learning → NLP & LLM → RNN, LSTM & Transformer









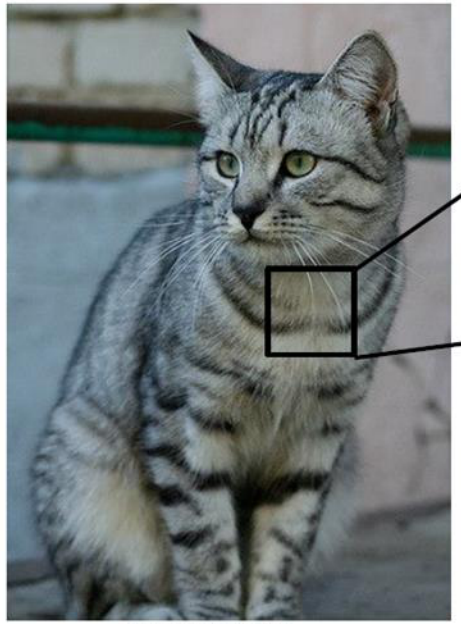


Agenda

- Introduction to convolutional layer
- Convolutional Neural Networks (CNN)
- Visual Transformer



Recall: The Problem: Semantic Gap



This image by Nikita is licensed under CC-BY 2.0

```
[[105 112 108 111 104  99 106  99  96 103 112 119 104  97  93  87]
 [ 91  98 102 106 104  79  98 103  99 105 123 136 110 105  94  85]
 [ 76  85  90 105 120 105  87  96  95  99 115 112 106 103  99  85]
 [ 99  81  81  93 120 131 127 100  95  98 102  99  96  93 101  94]
 [106  91  61  64  69  91  88  85 101 107 109  98  75  84  96  95]
 [114 108  85  55  55  69  64  54  64  87 112 129  98  74  84  91]
 [133 137 147 103  65  81  80  65  52  54  74  84 102  93  85  82]
 [128 137 144 140 109  95  86  70  62  65  63  63  60  73  86 101]
 [125 133 148 137 119 121 117  94  65  79  80  65  54  64  72  98]
 [127 125 131 147 133 127 126 131 111  96  89  75  61  64  72  84]
 [115 114 109 123 150 148 131 118 113 109 100  92  74  65  72  78]
 [ 89  93  90  97 108 147 131 118 113 114 113 109 106  95  77  80]
 [ 63  77  86  81  77  79 102 123 117 115 117 125 125 130 115  87]
 [ 62  65  82  89  78  71  80 101 124 126 119 101 107 114 131 119]
 [ 63  65  75  88  89  71  62  81 120 138 135 105  81  98 110 118]
 [ 87  65  71  87 106  95  69  45  76 130 126 107  92  94 105 112]
 [118  97  82  86 117 123 116  66  41  51  95  93  89  95 102 107]
 [164 146 112  80  82 120 124 104  76  48  45  66  88 101 102 109]
 [157 170 157 120  93  86 114 132 112  97  69  55  70  82  99  94]
 [130 128 134 161 139 100 109 118 121 134 114  87  65  53  69  86]
 [128 112  96 117 150 144 120 115 104 107 102  93  87  81  72  79]
 [123 107  96  86  83 112 153 149 122 109 104  75  80 107 112  99]
 [122 121 102  80  82  86  94 117 145 148 153 102  58  78  92 107]
 [122 164 148 103  71  56  78  83  93 103 119 139 102  61  69  84]]
```

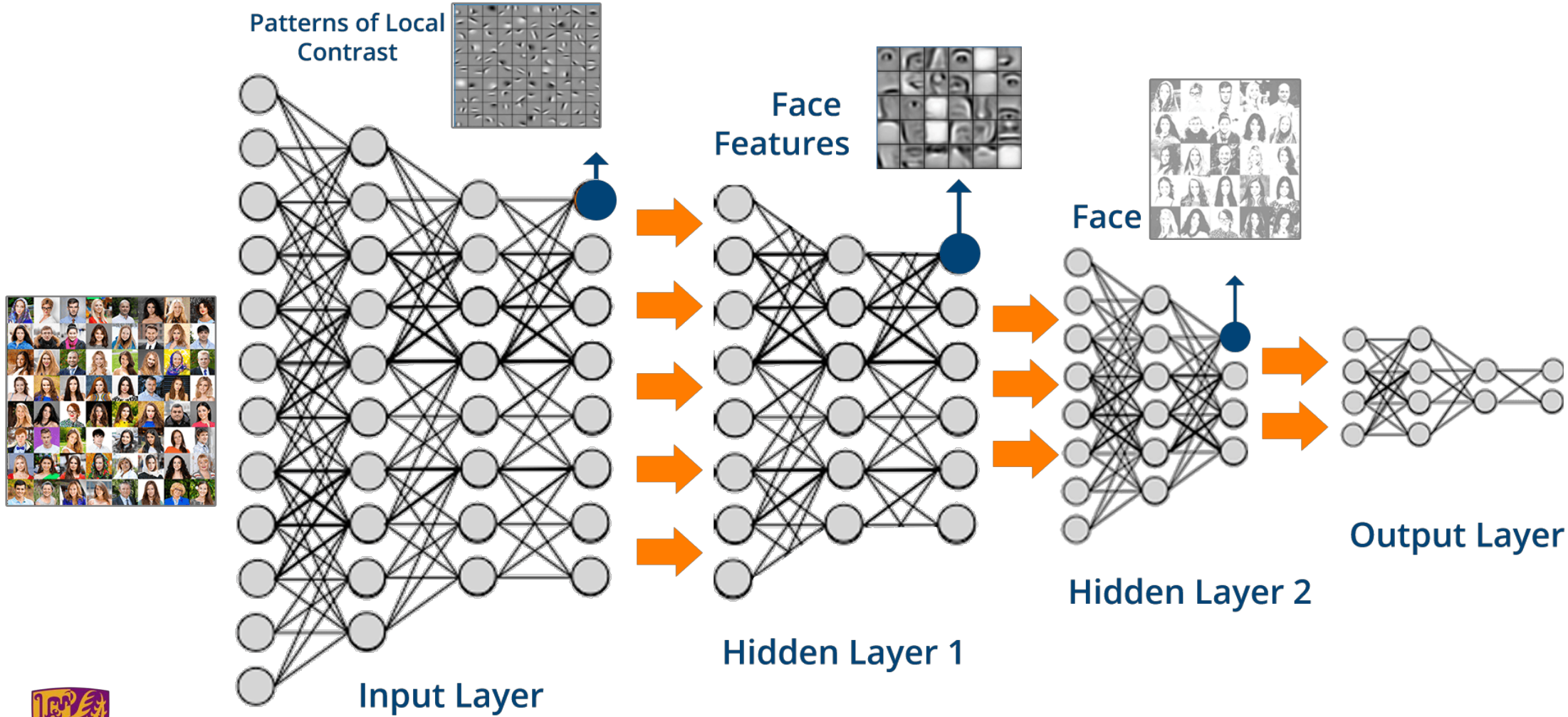
What the computer sees

An image is just a big grid of numbers between [0, 255]:

e.g. 800 x 600 x 3
(3 channels RGB)

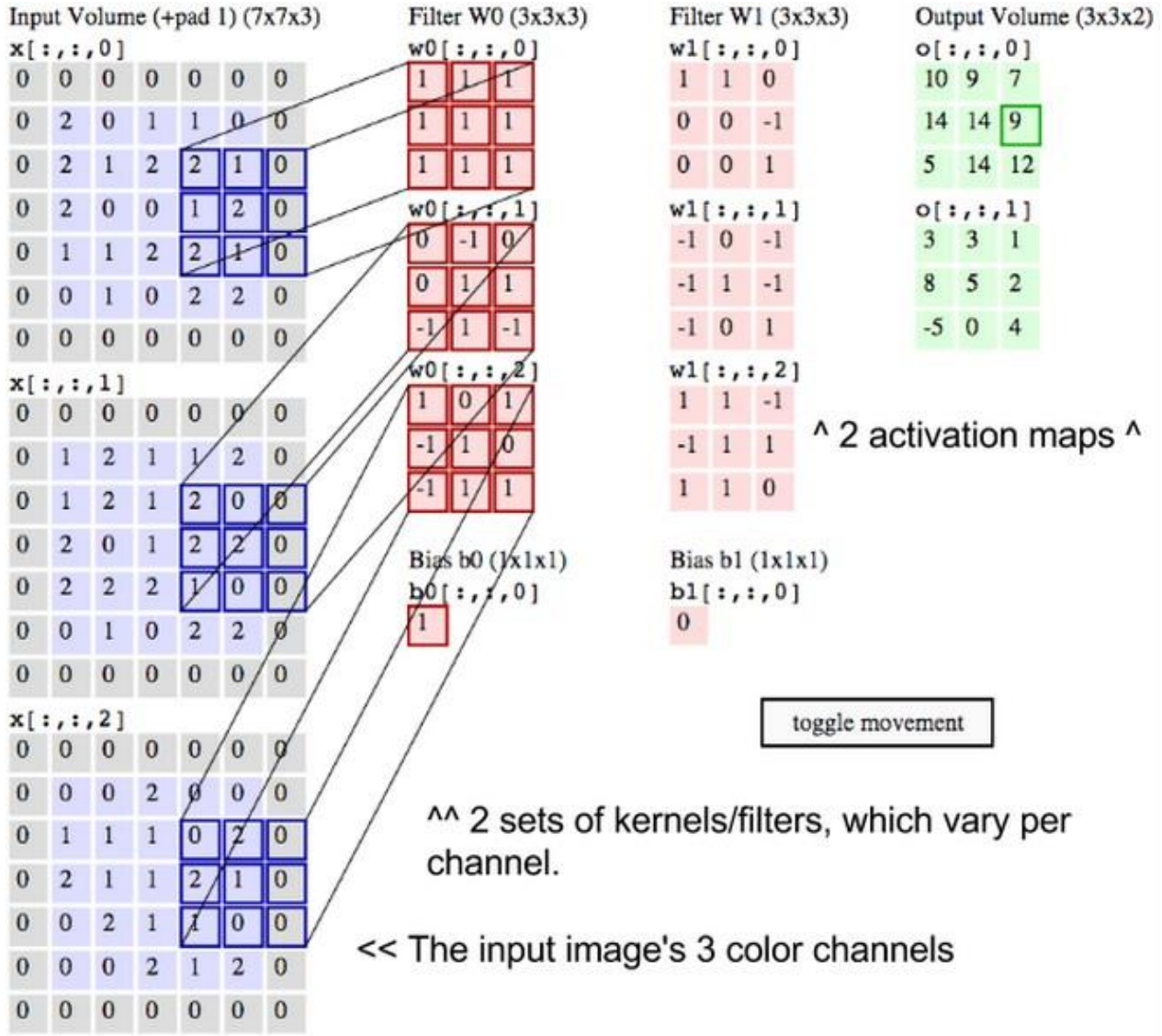
Examples: how computer classifies an image

Recall: In Era of Deep Learning



Convolutional Layer

- move the filter 1 pixel at a time from left to right and top to bottom





Why we want CNN?

- We know it is good to learn a small model.
- From this fully connected model, do we really need all the edges?
- Can some of these be shared?
- Can represent a small region with fewer parameters

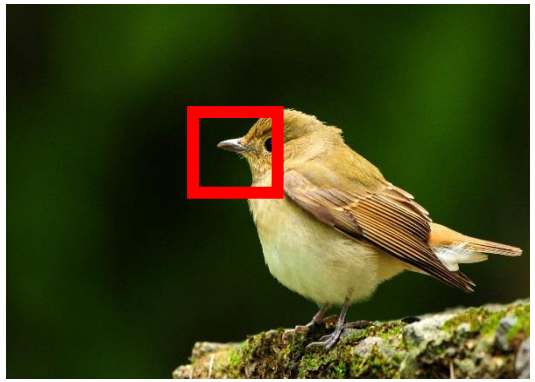


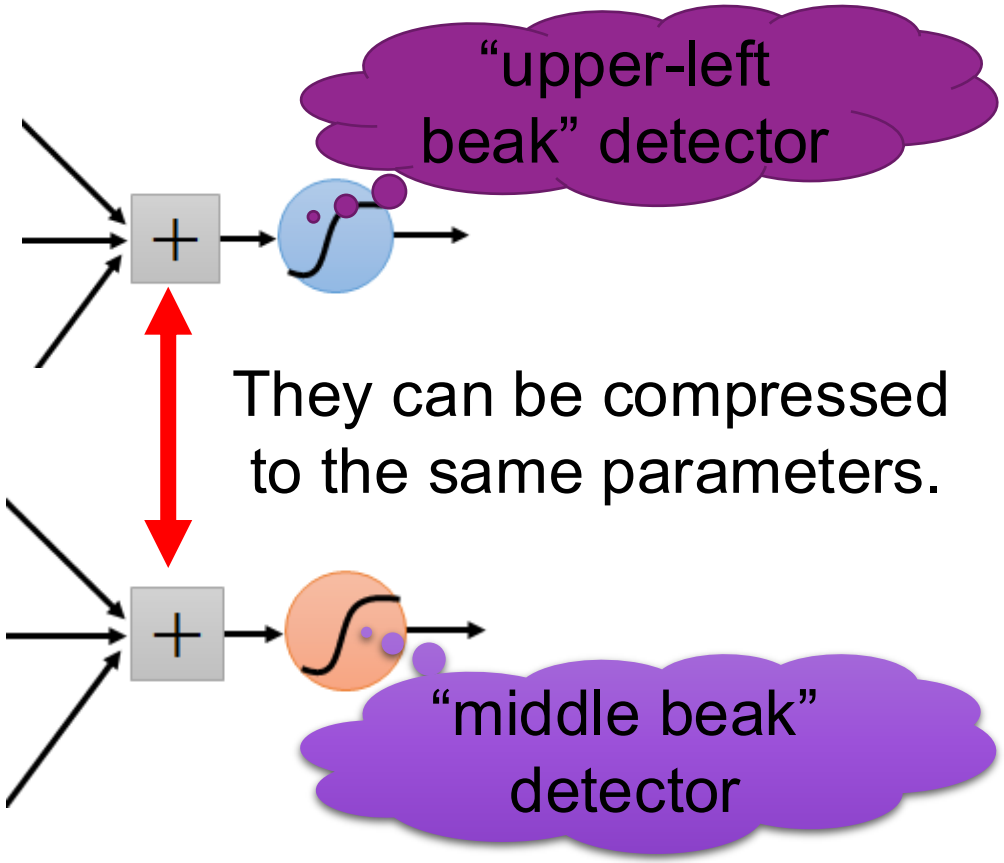


Same pattern appears in different places: They can be compressed!

- What about training a lot of such “small” detectors and each detector must “move around”.

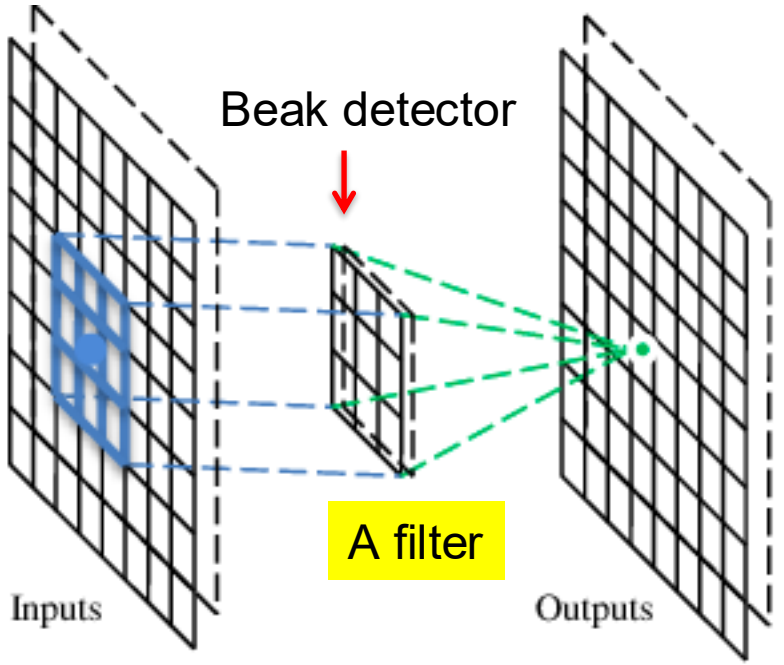






A convolutional layer

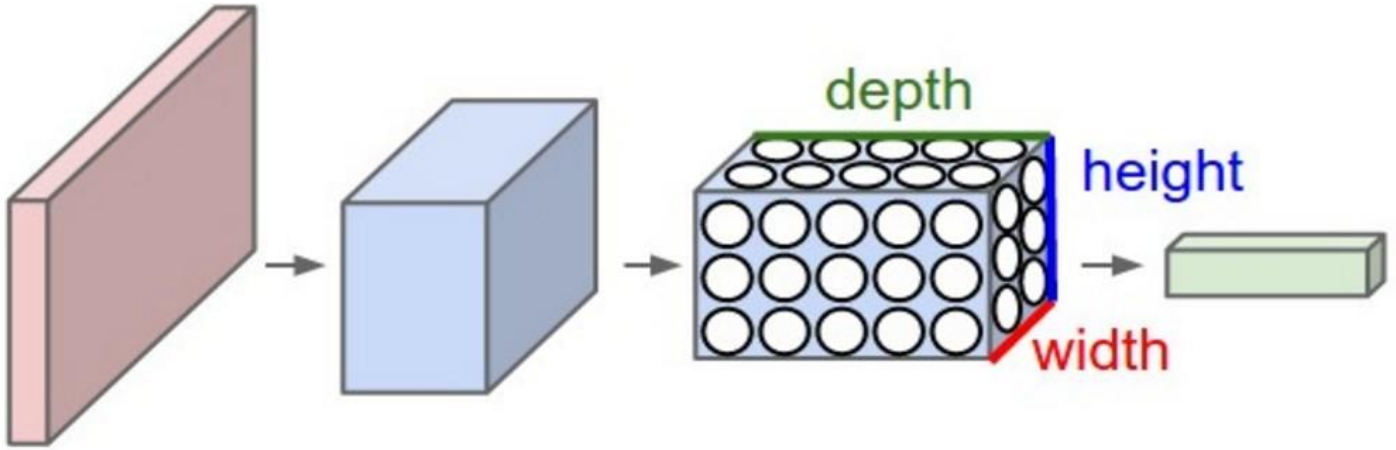
- A CNN is a neural network with some convolutional layers (and some other layers). A convolutional layer has a number of filters that does convolutional operation.





Convolutional Neural Networks (CNNs, ConvNets)

- A basic CNN consists of (will go through them 1 by 1):
  - **Convolutional Layer**: Sparse connections, shared weights
  - **Pooling Layer**: No weights
  - **Normalization layers**: Accelerate and stablize training
  - **Fully-Connected Layer**: same as the previous networks

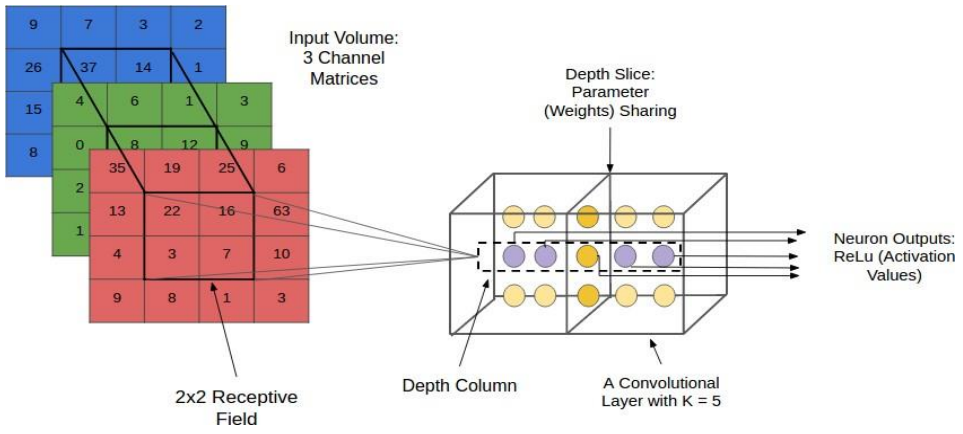


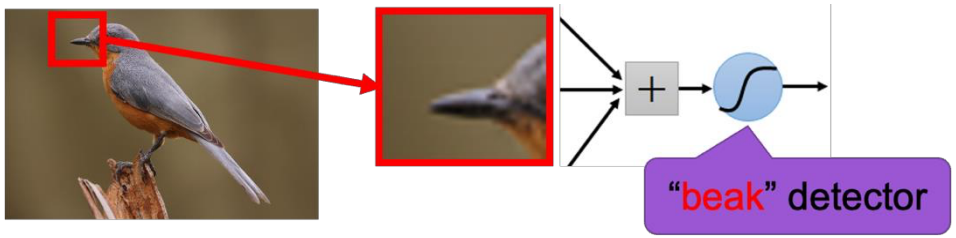


Convolutional Layer

- Connected only to units in a small patch (**receptive field**) of the previous layer
- The receptive field is local in space
- Always full along the entire depth (e.g., RGB) of the input

$$y[m,n] = x[m,n] * h[m,n] = \sum_{j=-\infty}^{\infty} \sum_{i=-\infty}^{\infty} x[i,j] \cdot h[m-i, n-j]$$





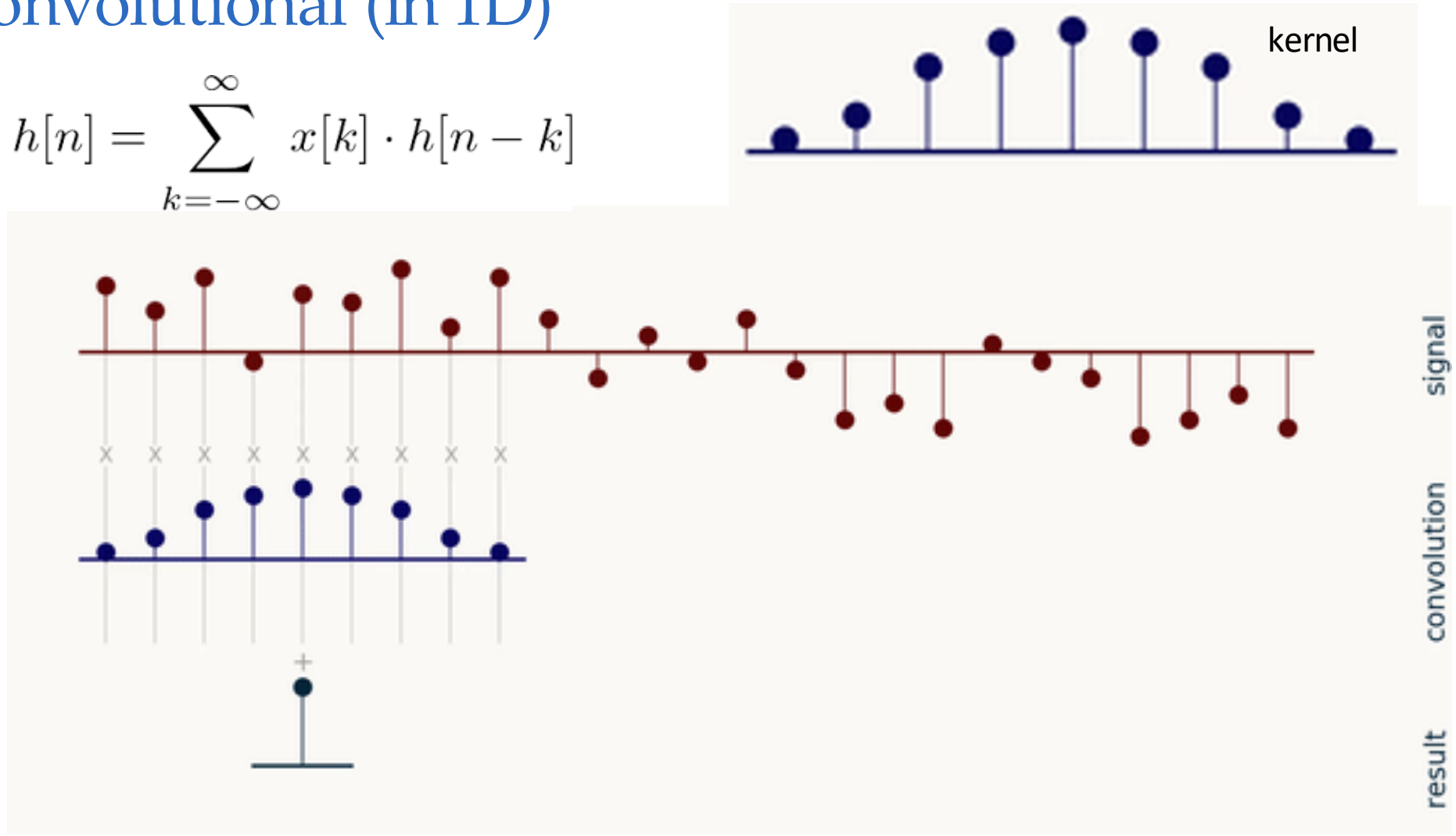
Convolutional Layer

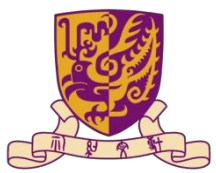
- The parameters are the connection weights and the bias to be learned
- A convolutional layer consists of a volume of convolutional units (or **filters**)
- A unit applies the same parameters to detect the features at different locations.
  - Hence, the set of weights is called a filter or kernel
  - Different units on the depth slices are obtained by sliding the **filters**



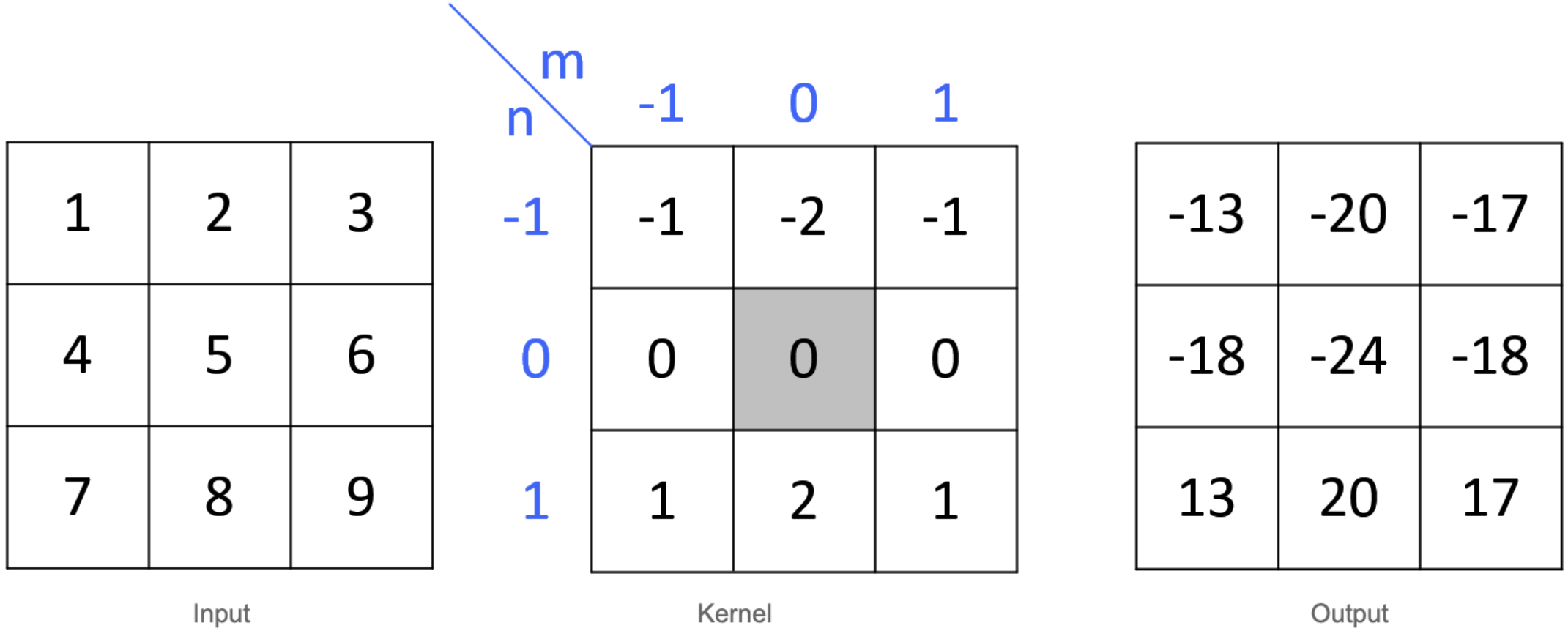
Recall: Convolutional (in 1D)

$$y[n] = x[n] * h[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n-k]$$

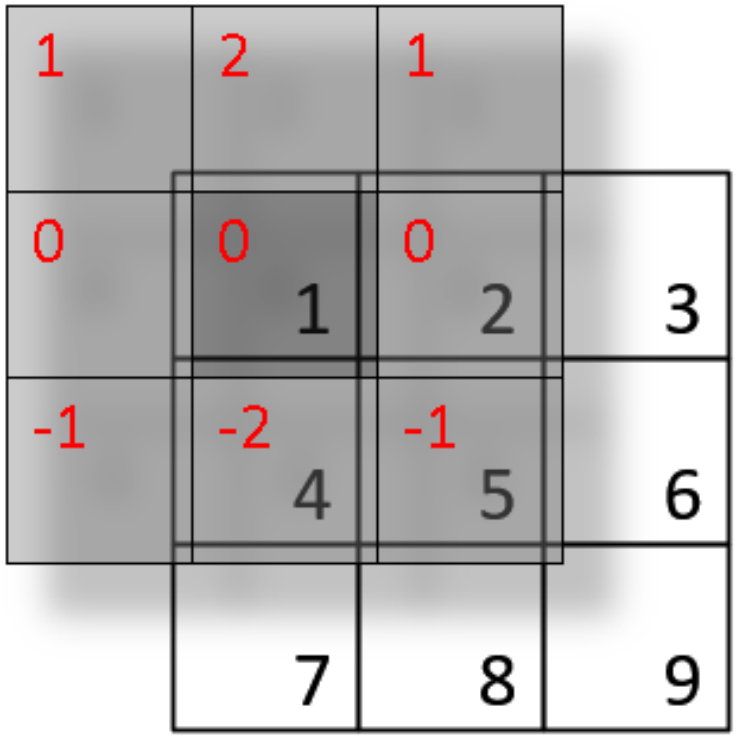




Convolution (in 2D): outputs

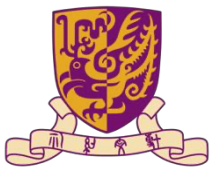


Convolution (in 2D)

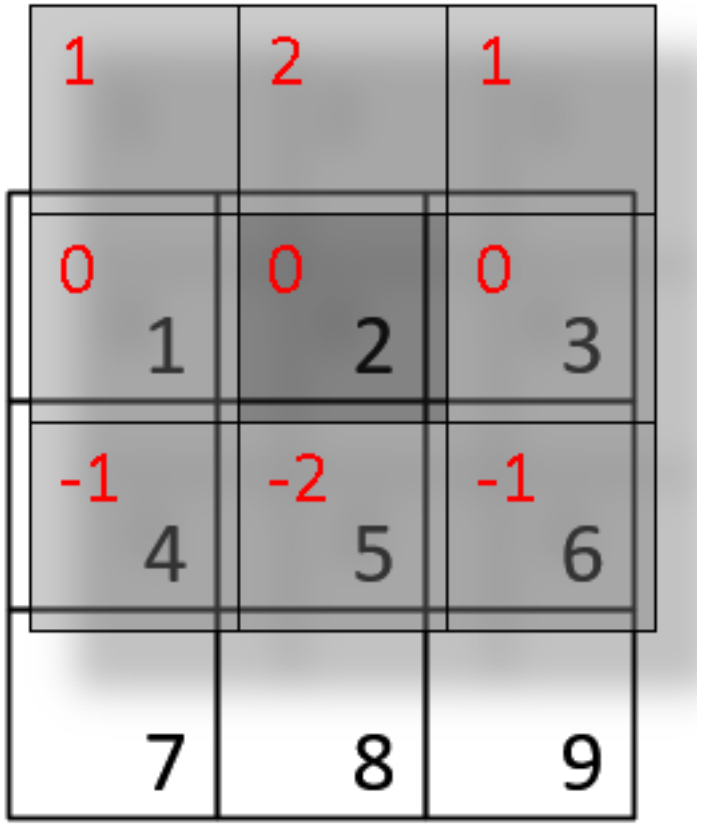


$$
\begin{aligned}
y[0,0] &= \sum_j \sum_i x[i,j] \cdot h[0-i, 0-j] \\
&= \quad x[-1,-1] \cdot h[1,1] + x[0,-1] \cdot h[0,1] + x[1,-1] \cdot h[-1,1] \\
&\quad + x[-1,0] \cdot h[1,0] \quad + x[0,0] \cdot h[0,0] \quad + x[1,0] \cdot h[-1,0] \\
&\quad + x[-1,1] \cdot h[1,-1] + x[0,1] \cdot h[0,-1] + x[1,1] \cdot h[-1,-1] \\
&= \quad 0 \cdot 1 + 0 \cdot 2 + 0 \cdot 1 \\
&\quad + 0 \cdot 0 + 1 \cdot 0 + 2 \cdot 0 \\
&\quad + 0 \cdot (-1) + 4 \cdot (-2) + 5 \cdot (-1) \\
&= -13
\end{aligned}
$$

$$
y[m,n] = x[m,n] * h[m,n] = \sum_{j=-\infty}^{\infty} \sum_{i=-\infty}^{\infty} x[i,j] \cdot h[m-i, n-j]
$$



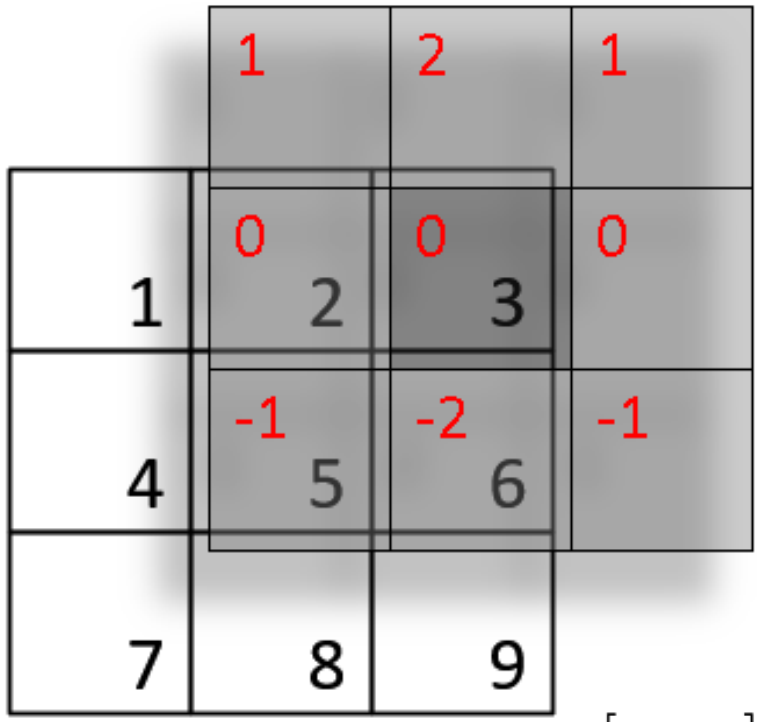
Convolution (in 2D)



$$
\begin{aligned}
y[1,0] &= \sum_j \sum_i x[i,j] \cdot h[1-i, 0-j] \\
&= \quad x[0,-1] \cdot h[1,1] + x[1,-1] \cdot h[0,1] + x[2,-1] \cdot h[-1,1] \\
&\quad + x[0,0] \cdot h[1,0] \quad + x[1,0] \cdot h[0,0] \quad + x[2,0] \cdot h[-1,0] \\
&\quad + x[0,1] \cdot h[1,-1] + x[1,1] \cdot h[0,-1] + x[2,1] \cdot h[-1,-1] \\
&= \quad 0 \cdot 1 + 0 \cdot 2 + 0 \cdot 1 \\
&\quad + 1 \cdot 0 + 2 \cdot 0 + 3 \cdot 0 \\
&\quad + 4 \cdot (-1) + 5 \cdot (-2) + 6 \cdot (-1) \\
&= -20
\end{aligned}
$$

$$
y[m,n] = x[m,n] * h[m,n] = \sum_{j=-\infty}^{\infty} \sum_{i=-\infty}^{\infty} x[i,j] \cdot h[m-i, n-j]
$$

Convolution (in 2D)



$$y[2,0] = \sum_j \sum_i x[i,j] \cdot h[2-i, 0-j]$$

$$\begin{aligned}
&= \quad x[1,-1] \cdot h[1,1] + x[2,-1] \cdot h[0,1] + x[3,-1] \cdot h[-1,1] \\
&\quad + x[1,0] \cdot h[1,0] \quad + x[2,0] \cdot h[0,0] \quad + x[3,0] \cdot h[-1,0] \\
&\quad + x[1,1] \cdot h[1,-1] + x[2,1] \cdot h[0,-1] + x[3,1] \cdot h[-1,-1] \\
&= \quad 0 \cdot 1 + 0 \cdot 2 + 0 \cdot 1 \\
&\quad + 2 \cdot 0 + 3 \cdot 0 + 0 \cdot 0 \\
&\quad + 5 \cdot (-1) + 6 \cdot (-2) + 0 \cdot (-1) \\
&= -17
\end{aligned}$$

$$y[m,n] = x[m,n] * h[m,n] = \sum_{j=-\infty}^{\infty} \sum_{i=-\infty}^{\infty} x[i,j] \cdot h[m-i, n-j]$$

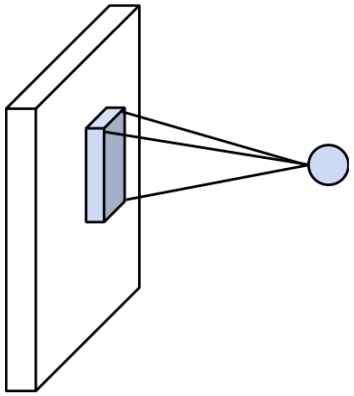
Agenda

- Introduction to convolutional layer
- Convolutional Neural Networks (CNN)
- Visual Transformer

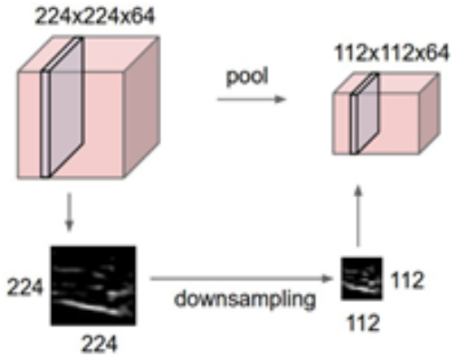


Components of CNNs

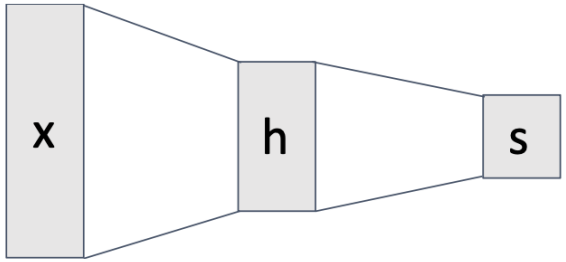
Convolution Layers



Pooling Layers



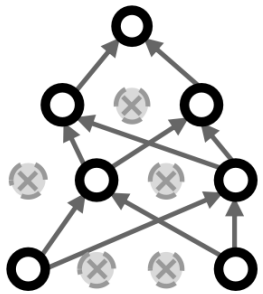
Fully-Connected Layers



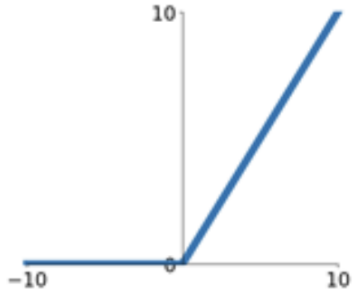
Normalization Layers

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

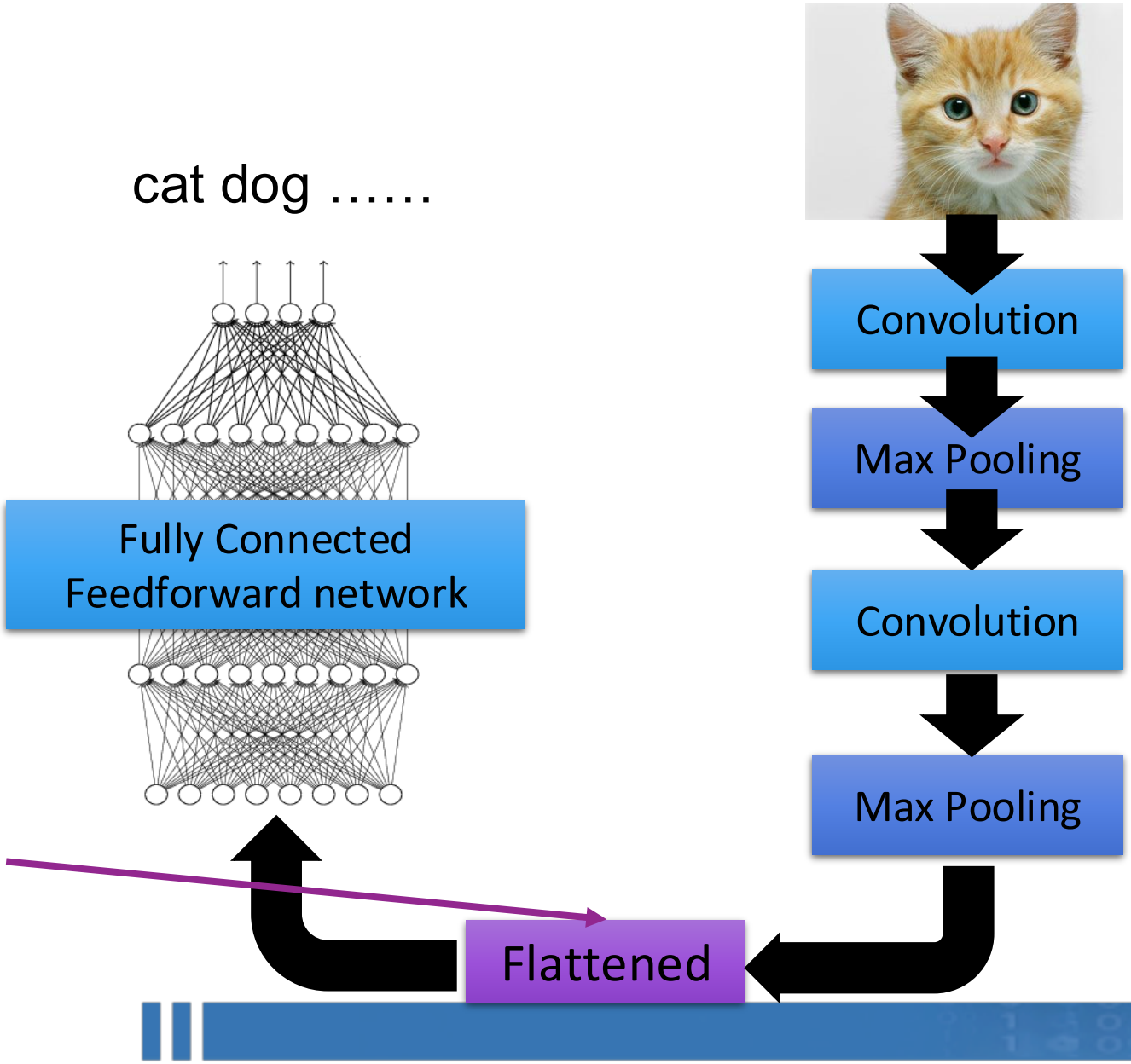
Dropout (sometimes)



Activation Functions



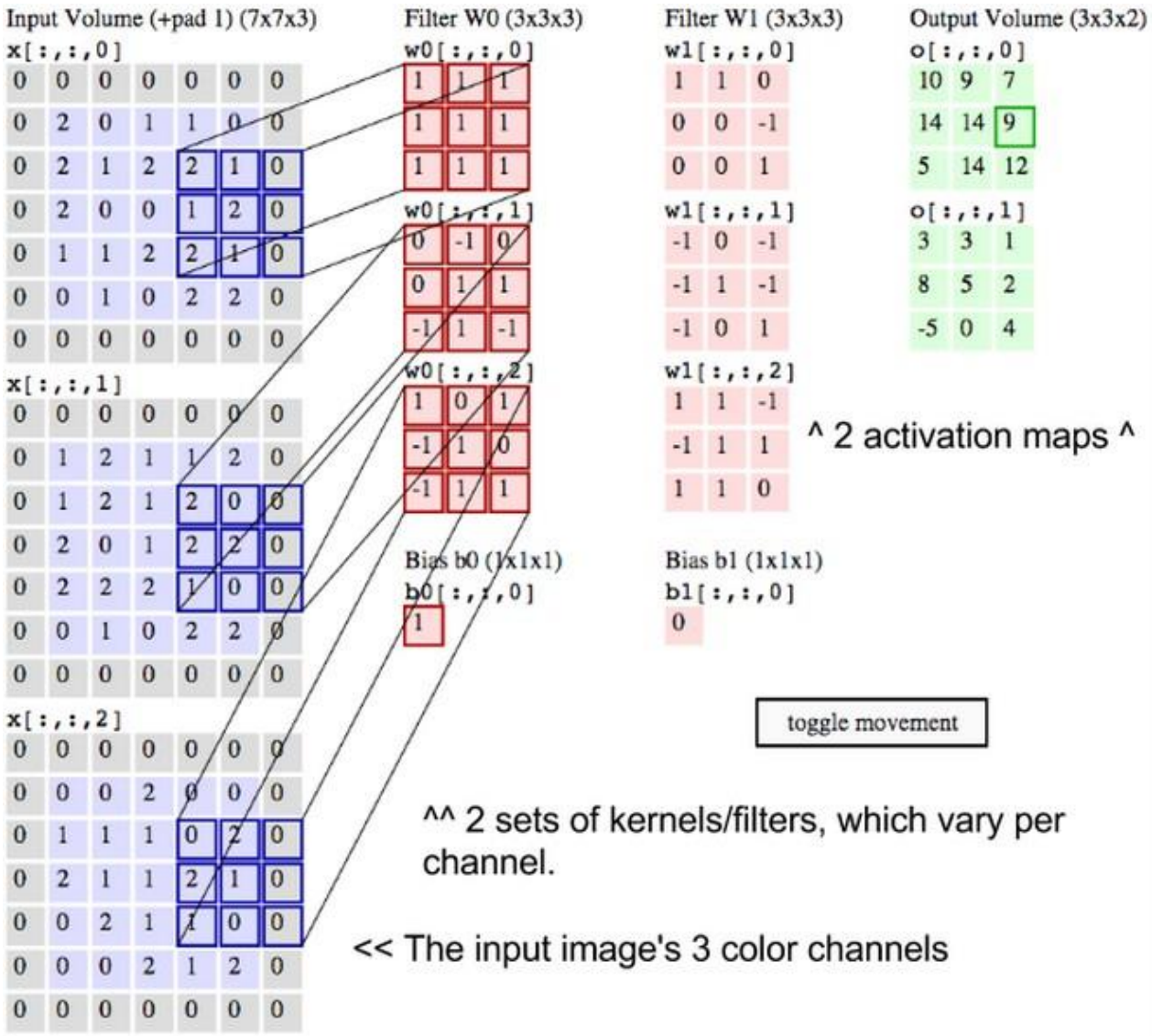
Overall



This is the layer for Transfer learning

Convolutional neural network

- move the filter 1 pixel at a time from left to right and top to bottom



Convolutional Layer

These are the network parameters to be learned.

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

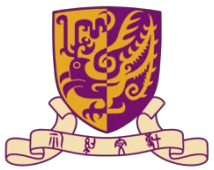
Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

⋮ ⋮

Each filter detects a small pattern (3 x 3).



Convolutional Layer

1	-1	-1
-1	1	-1
-1	-1	1

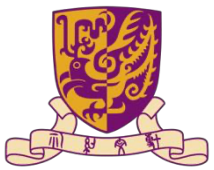
Filter 1

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Dot product → 3 -1

6 x 6 image



Convolutional Layer

1	-1	-1
-1	1	-1
-1	-1	1

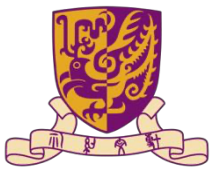
Filter 1

If stride=2

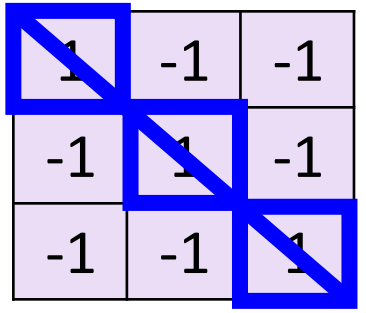
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

3 -3

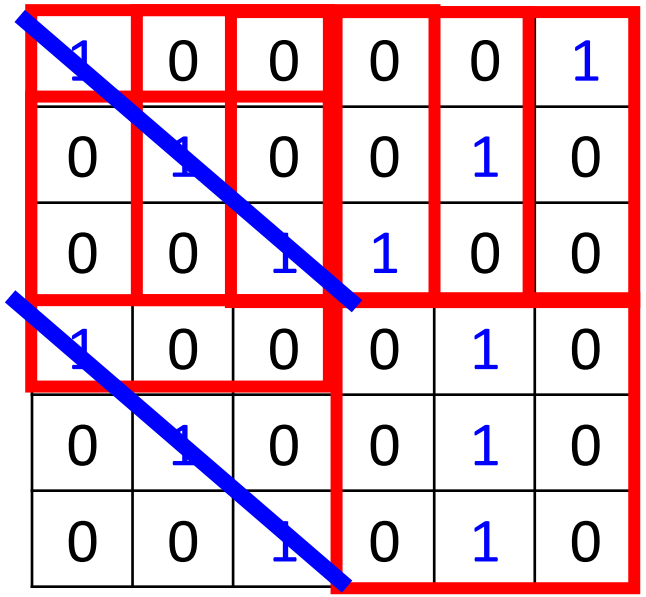


Convolutional Layer

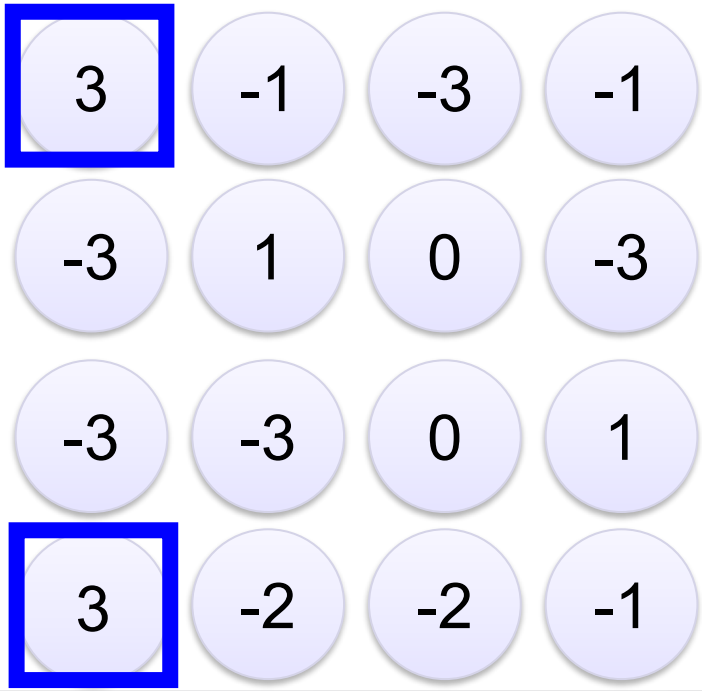


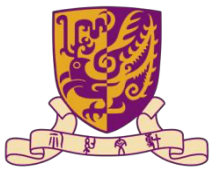
Filter 1

stride=1



6 x 6 image





Convolutional neural network

-1	1	-1
-1	1	-1
-1	1	-1

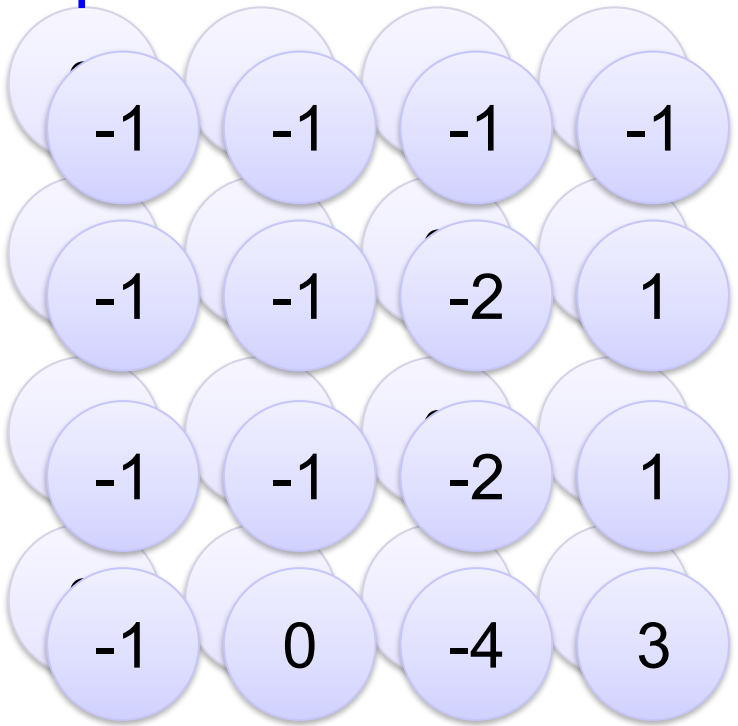
Filter 2

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

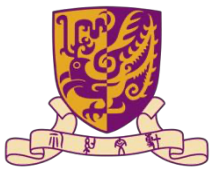
6 x 6 image

Repeat this for each filter



Feature Map

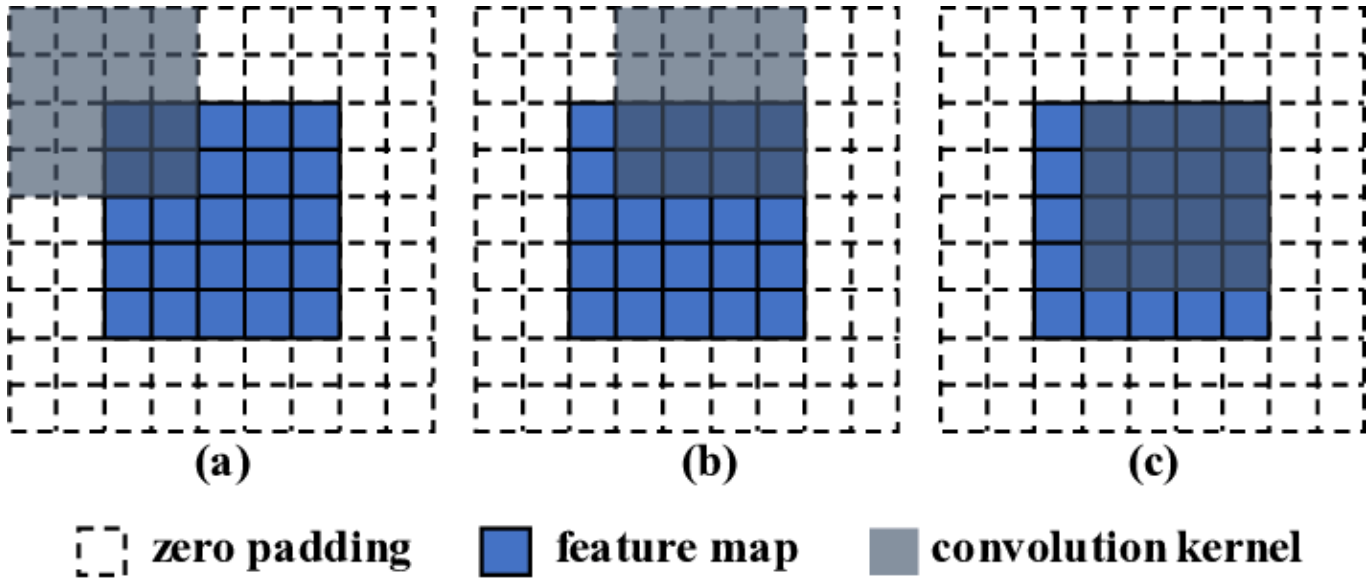
Two 4 x 4 images: Forming 2 x 4 x 4 matrix



Zero Padding

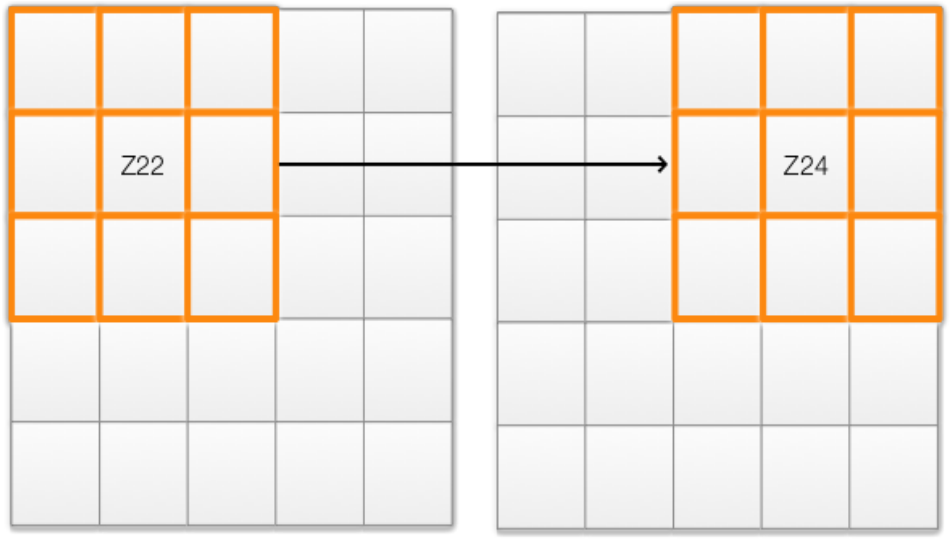
- better preserve information on the edge.

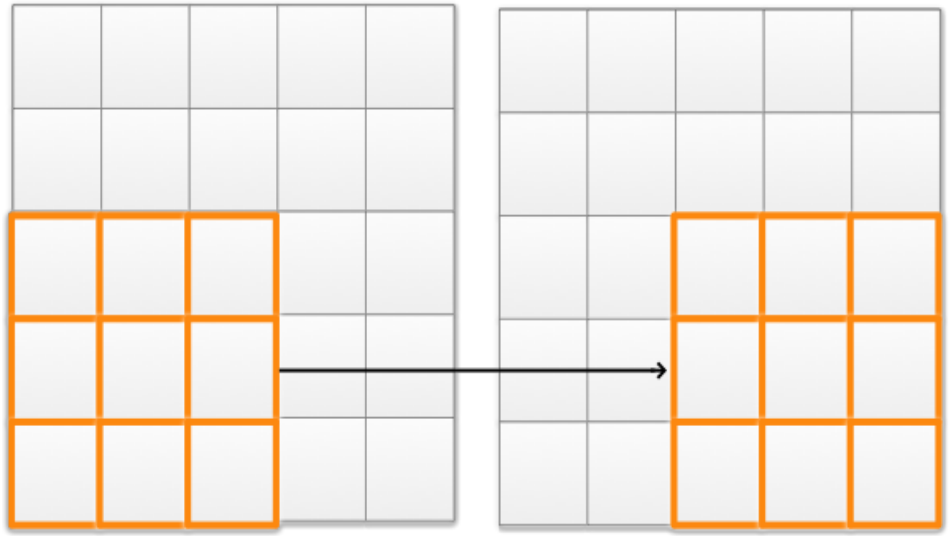
- maintain the spatial dimensions by packing extra 0 or replicate the edge of the original image



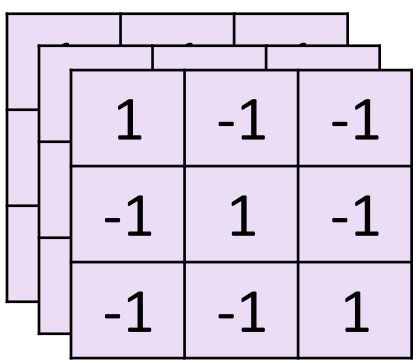
Stride

- move the filter 2 pixels each time, e.g., **stride** 2.
- It is uncommon to use stride 3 or more.

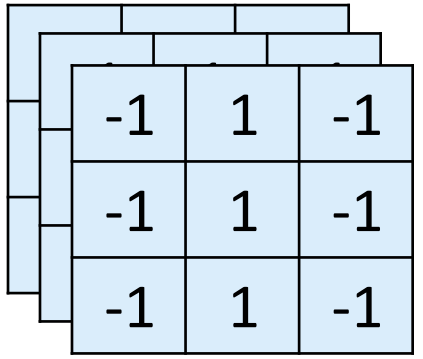




Color image: RGB 3 channels

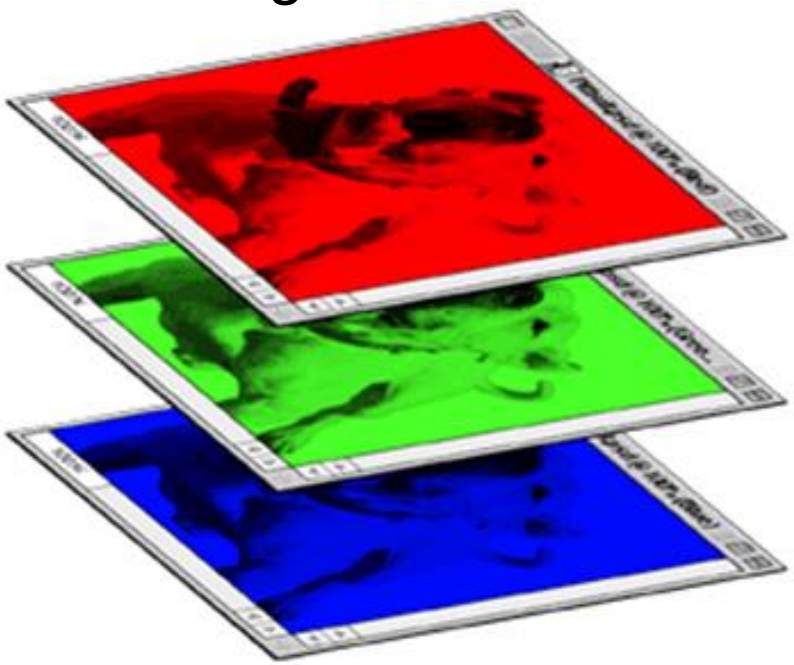


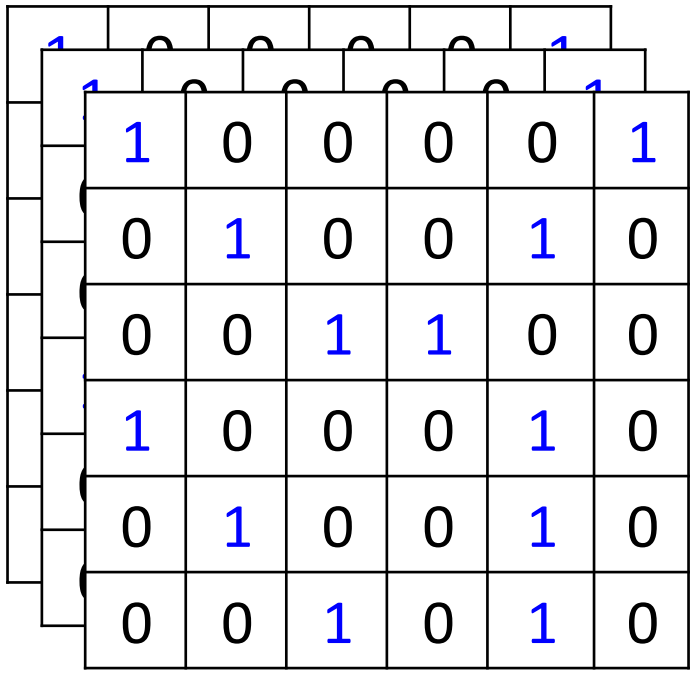
Filter 1



Filter 2

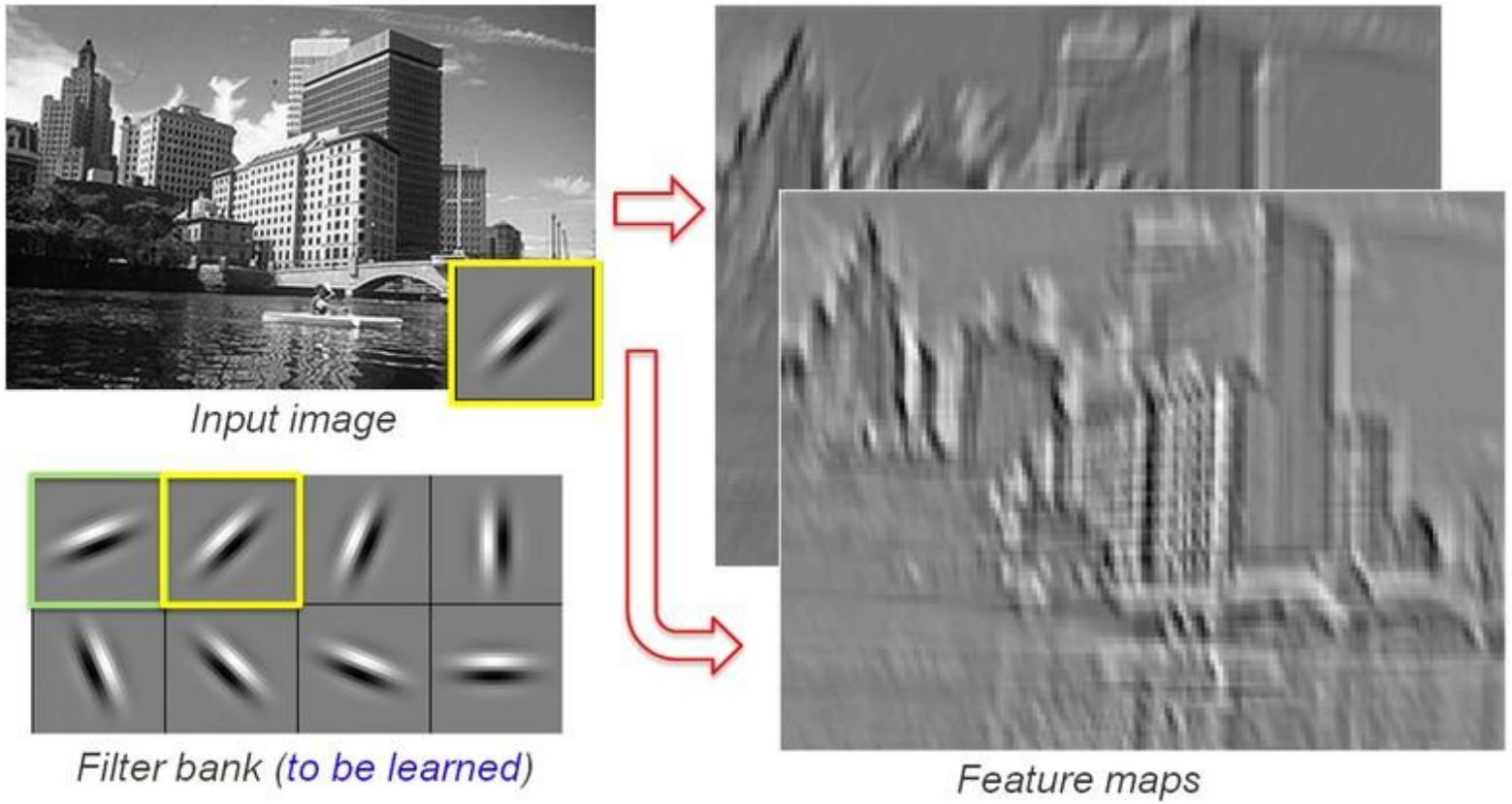
Color image





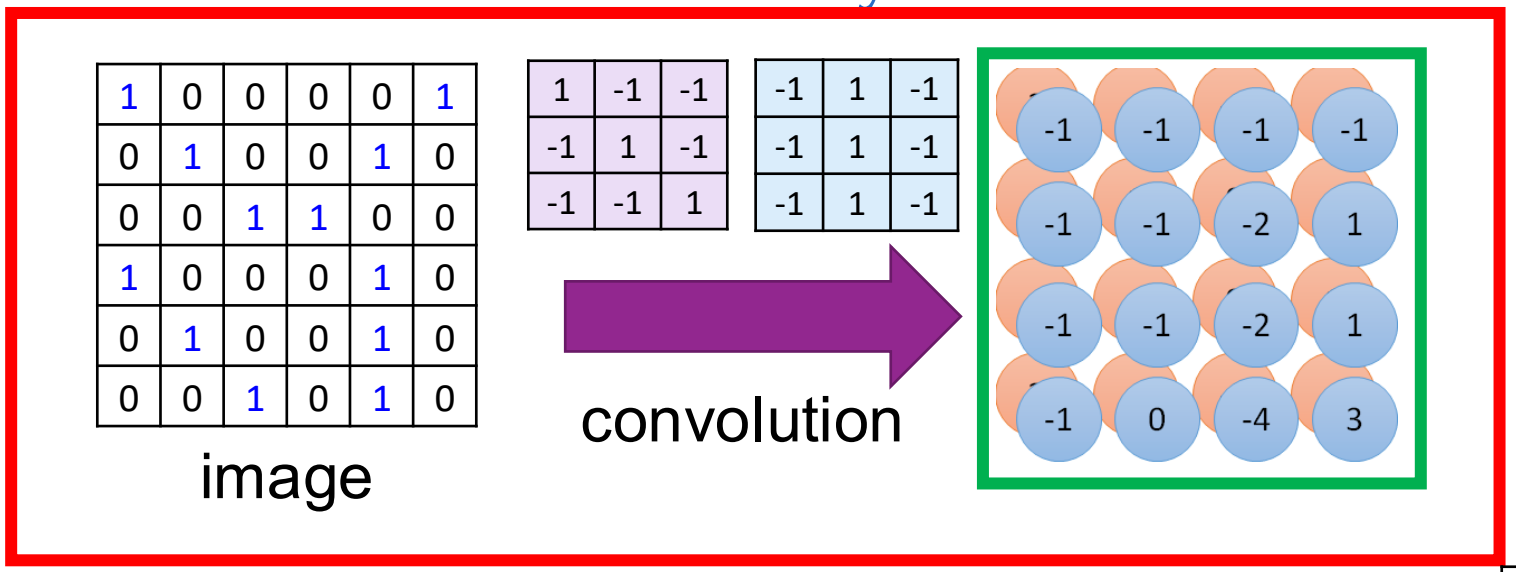
Convolutional Layer

- The output of each depth slice is called a **feature map.**

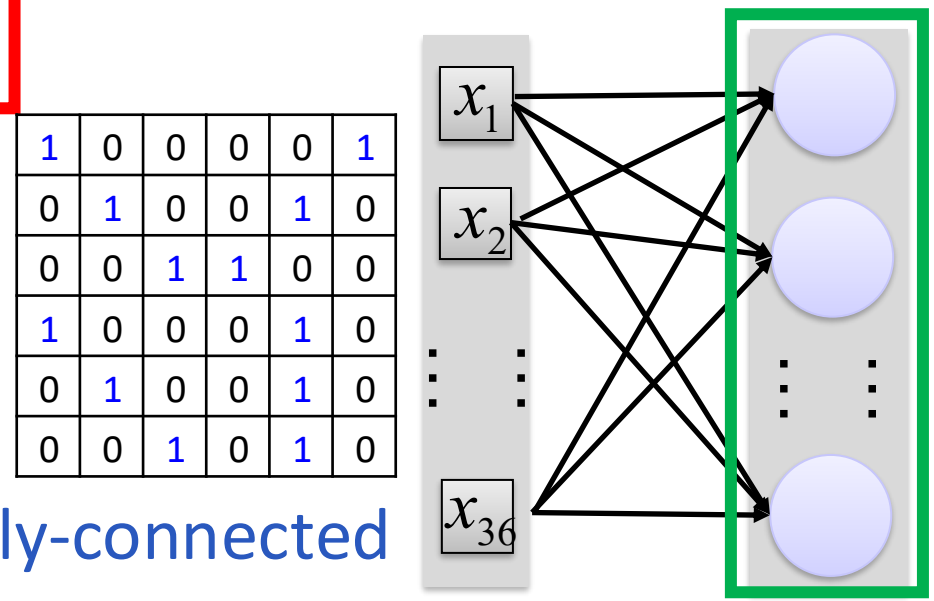


To be processed by the next layer

Convolution vs. Fully Connected



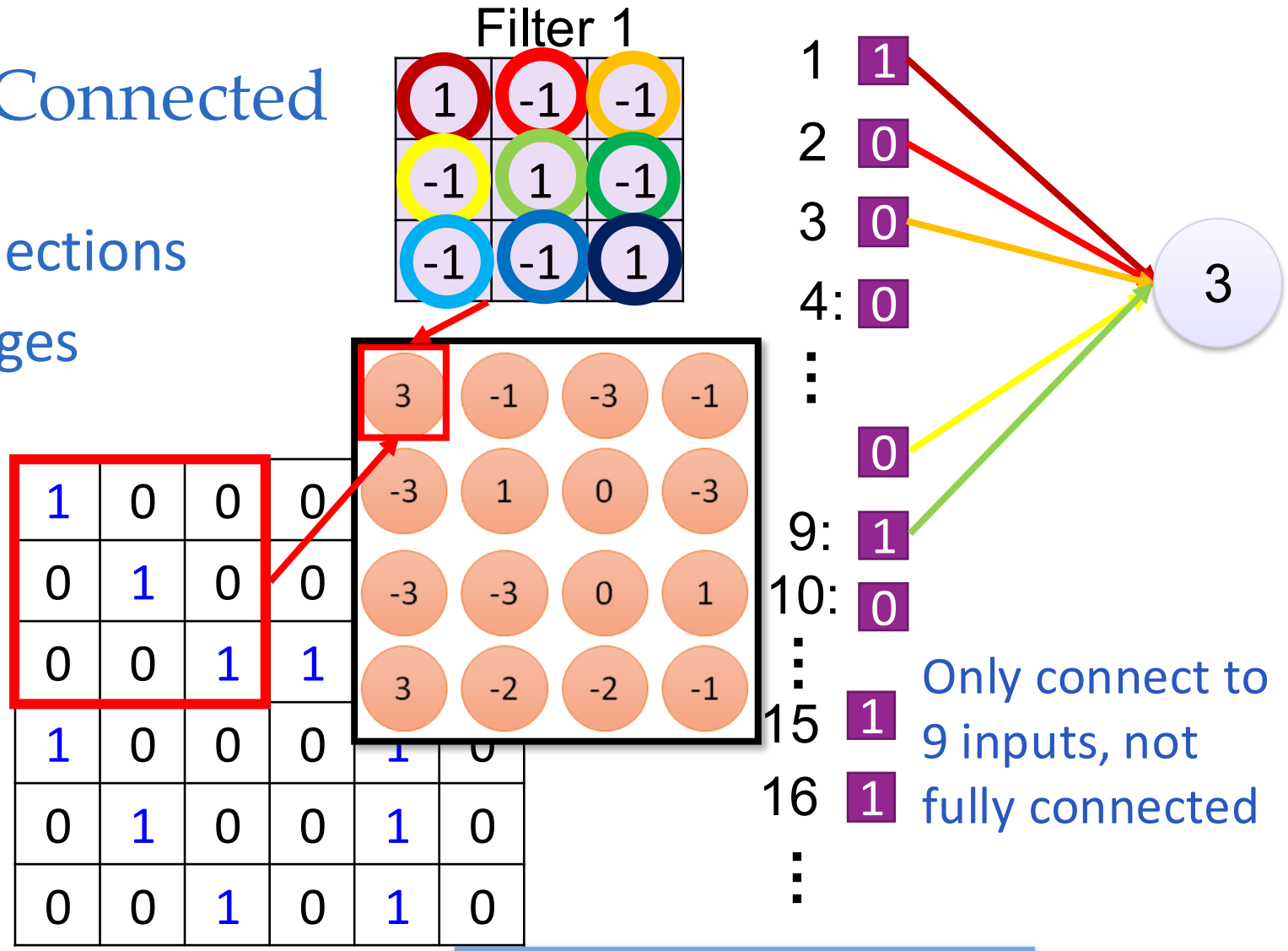
- Reducing number of connections
- Shared weights on the edges



Fully-connected

Convolution vs. Fully Connected

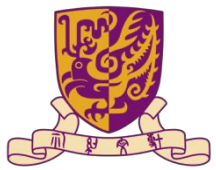
- Reducing number of connections
- Shared weights on the edges



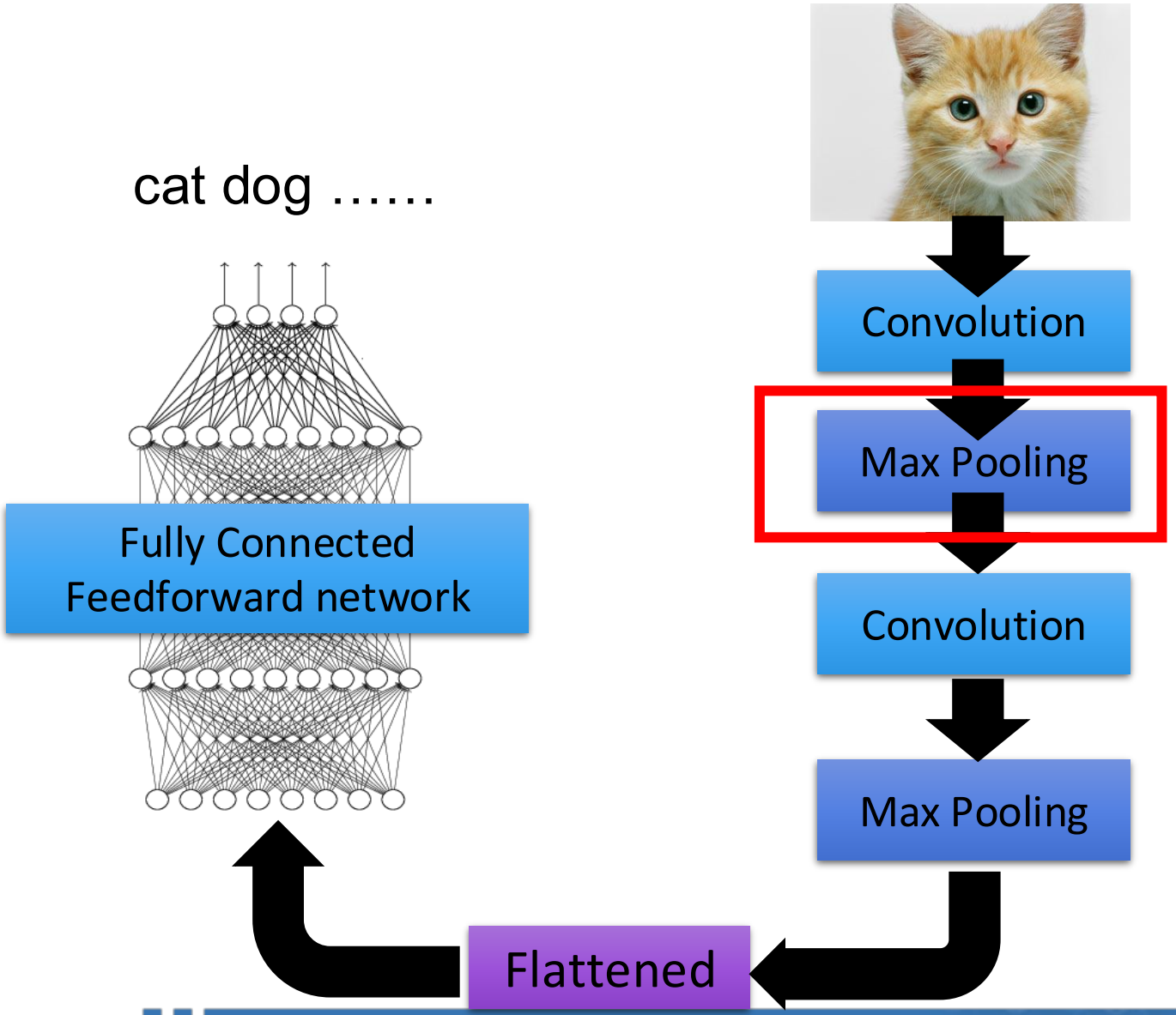
Recall: Why we want CNN?

- We know it is good to learn a small model.
- From this fully connected model, do we really need all the edges?
- Can some of these be shared?
- Can represent a small region with fewer parameters



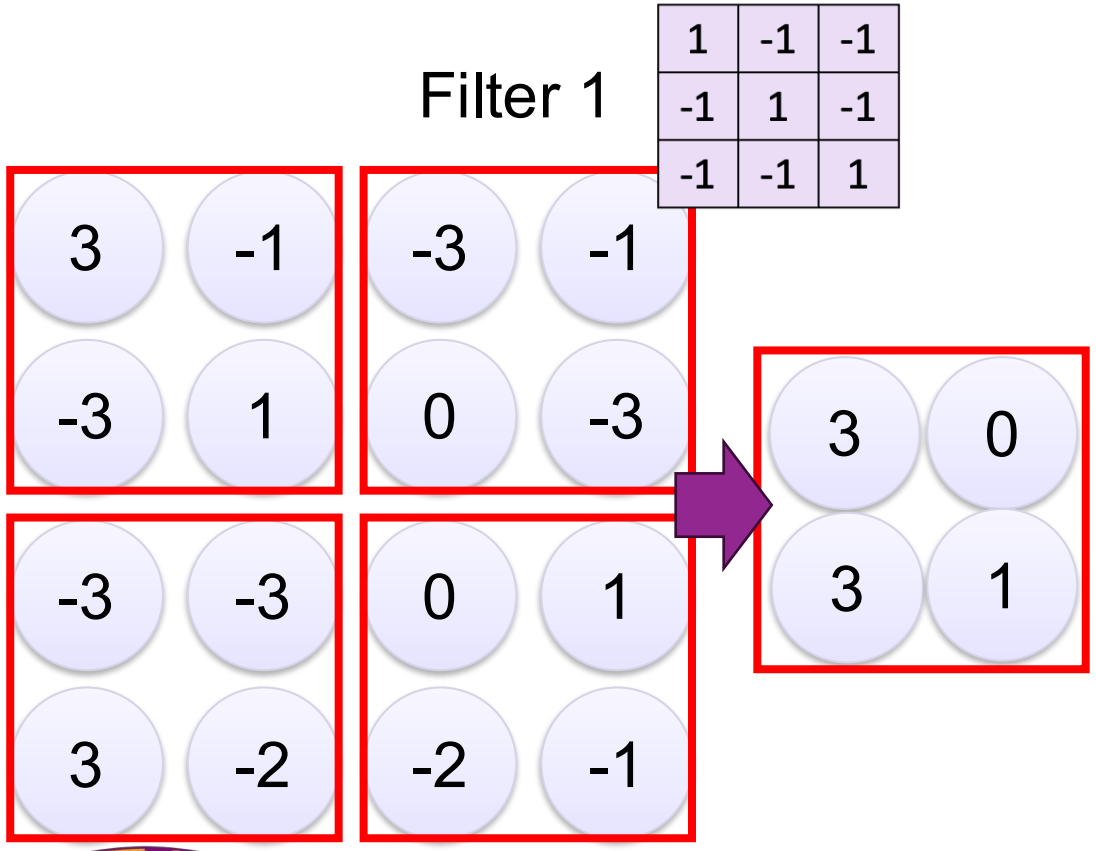


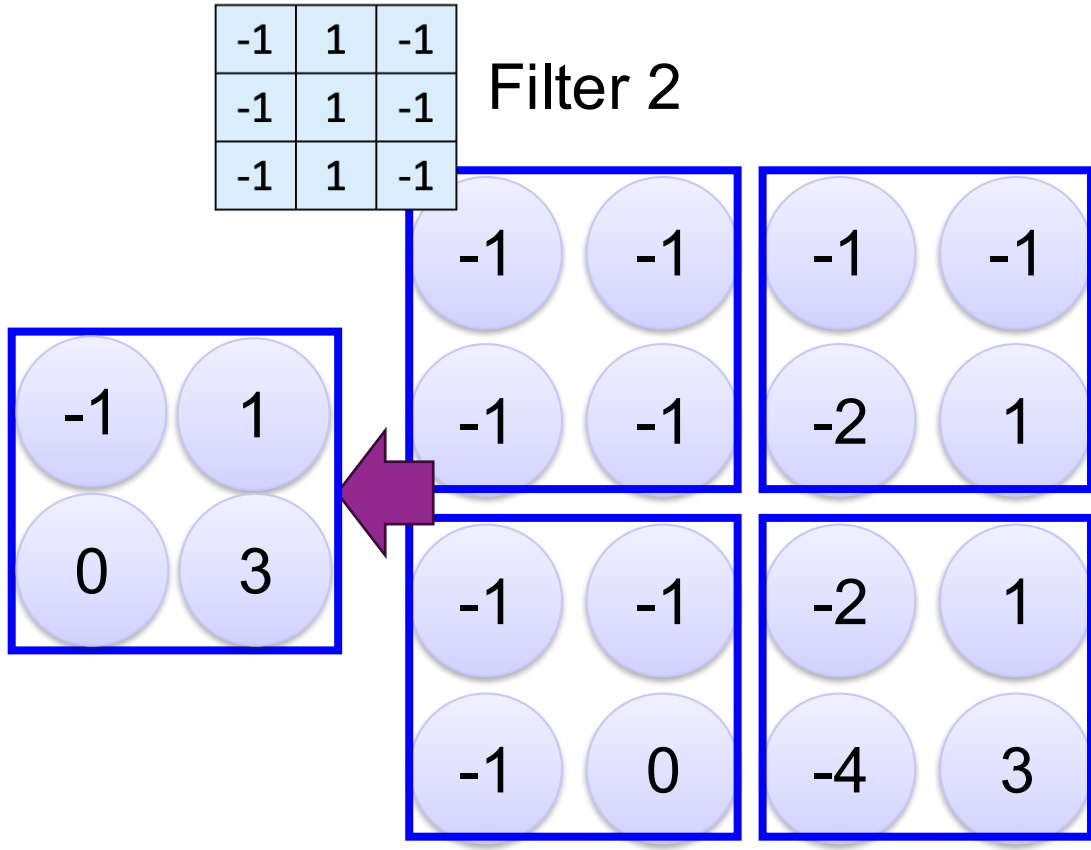
Overall


cat dog ……
Fully Connected Feedforward network
Convolution
Max Pooling
Convolution
Max Pooling
Flattened

Max Pooling

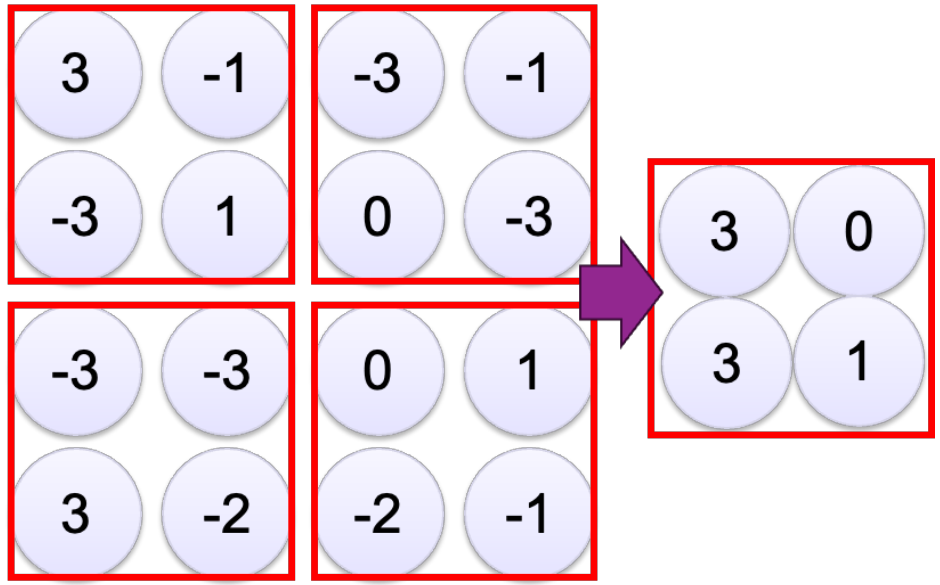
- Max pooling further reduces the complexity

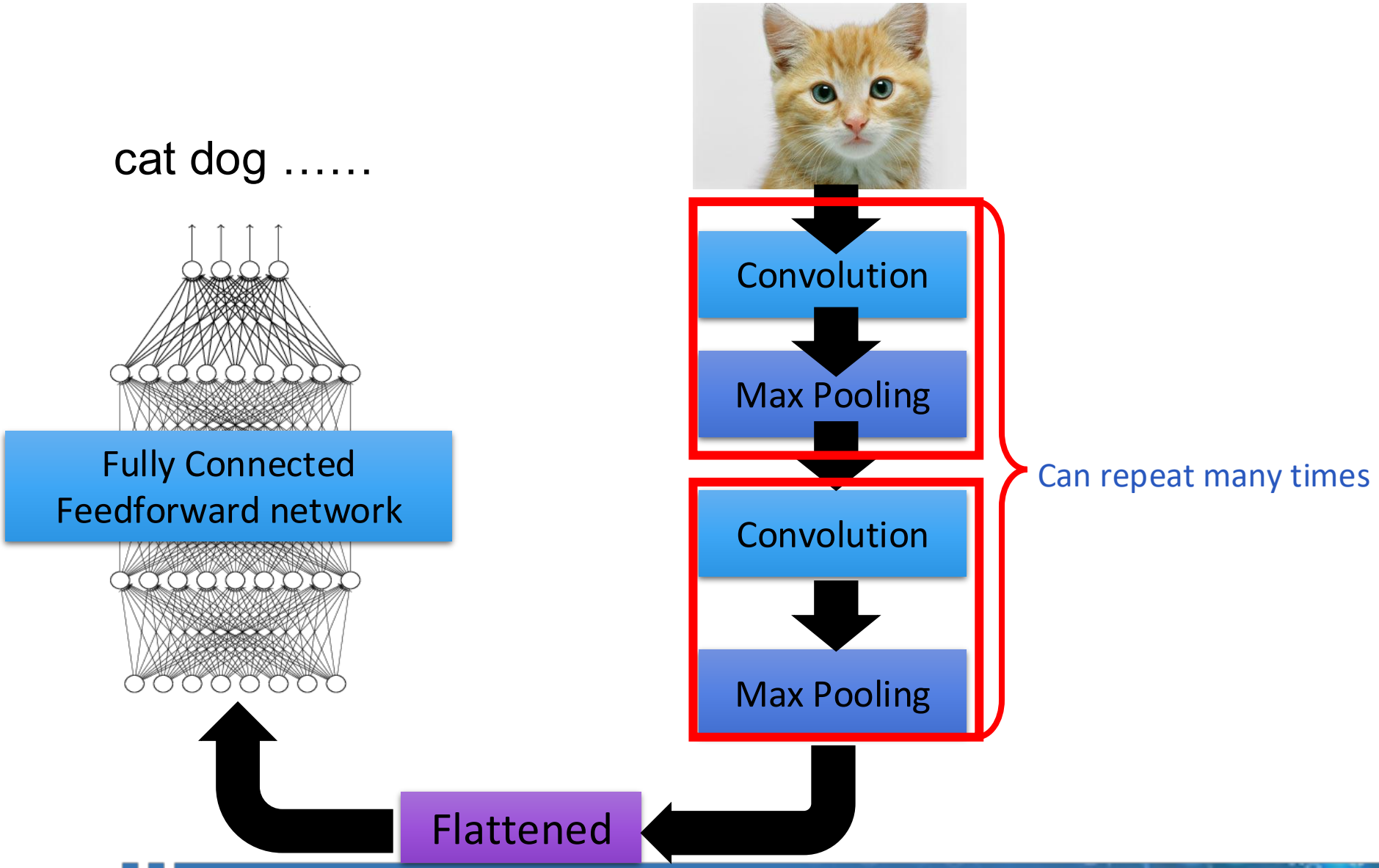


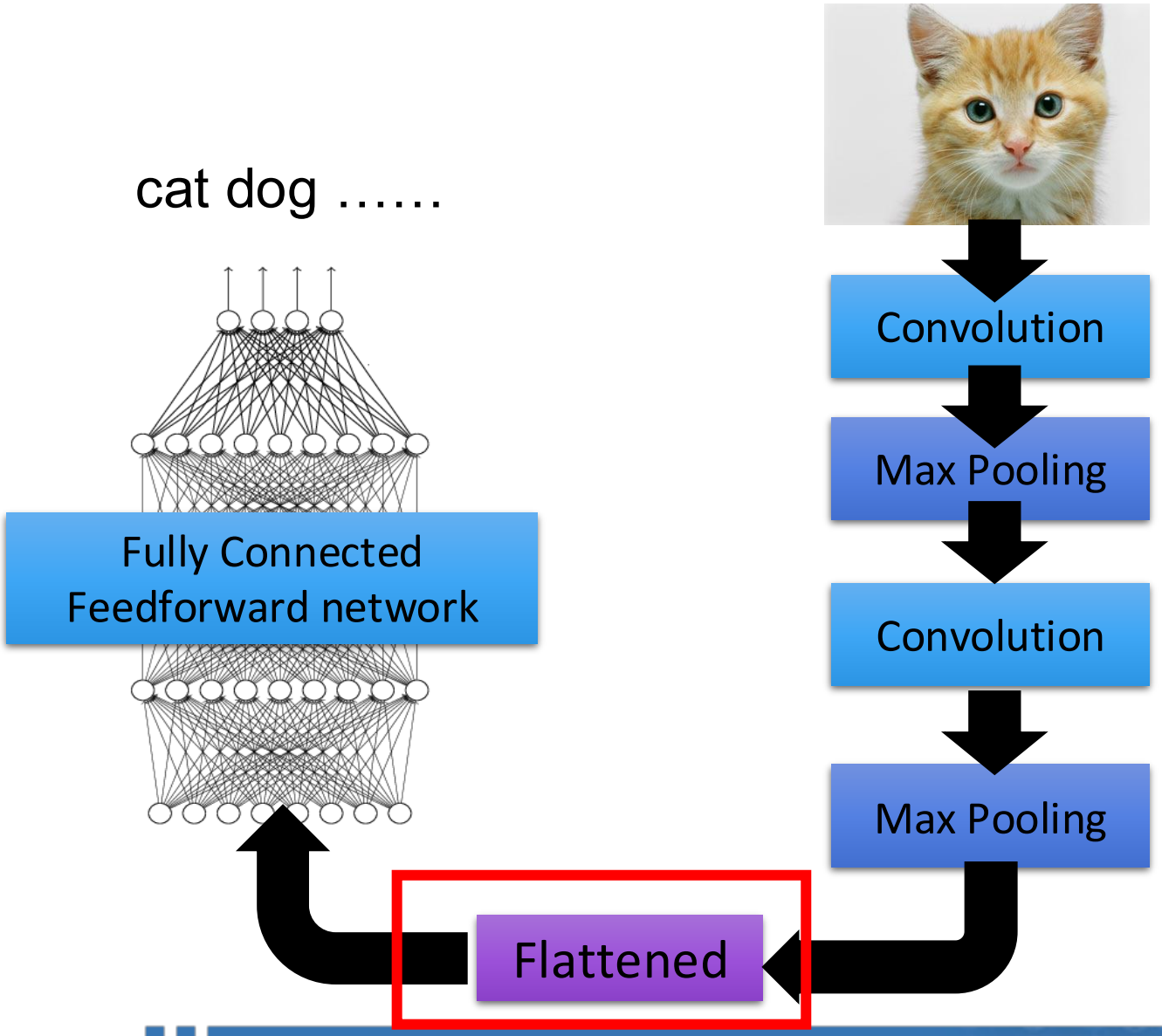


Why Pooling

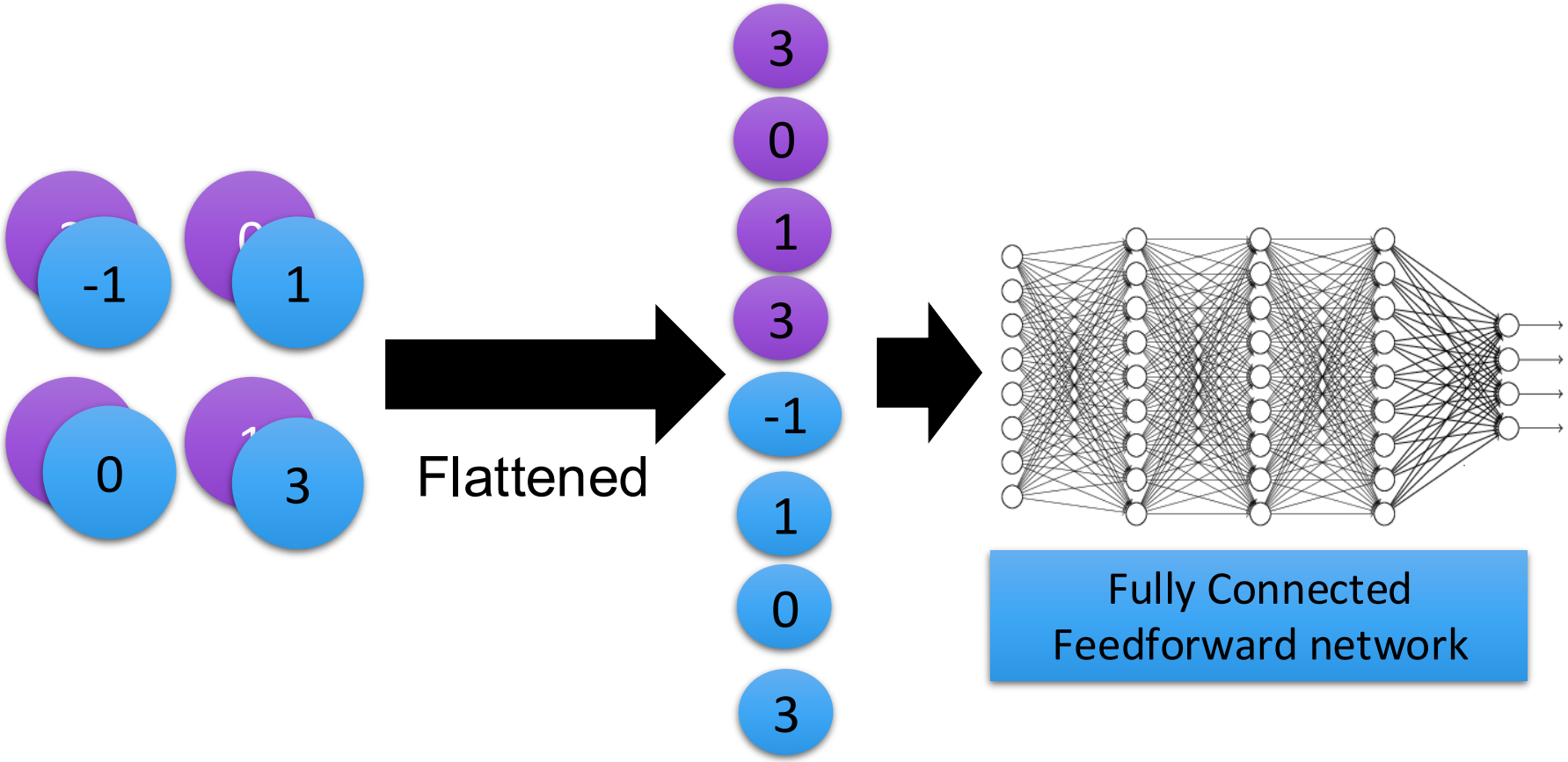
- Downsampling: Reduces dimensions while preserving features
- Translation Invariance: Recognizes patterns regardless of their location
- Feature Extraction: Selects the most prominent features
- Noise Robustness: Provides some resistance to noise or small perturbations.

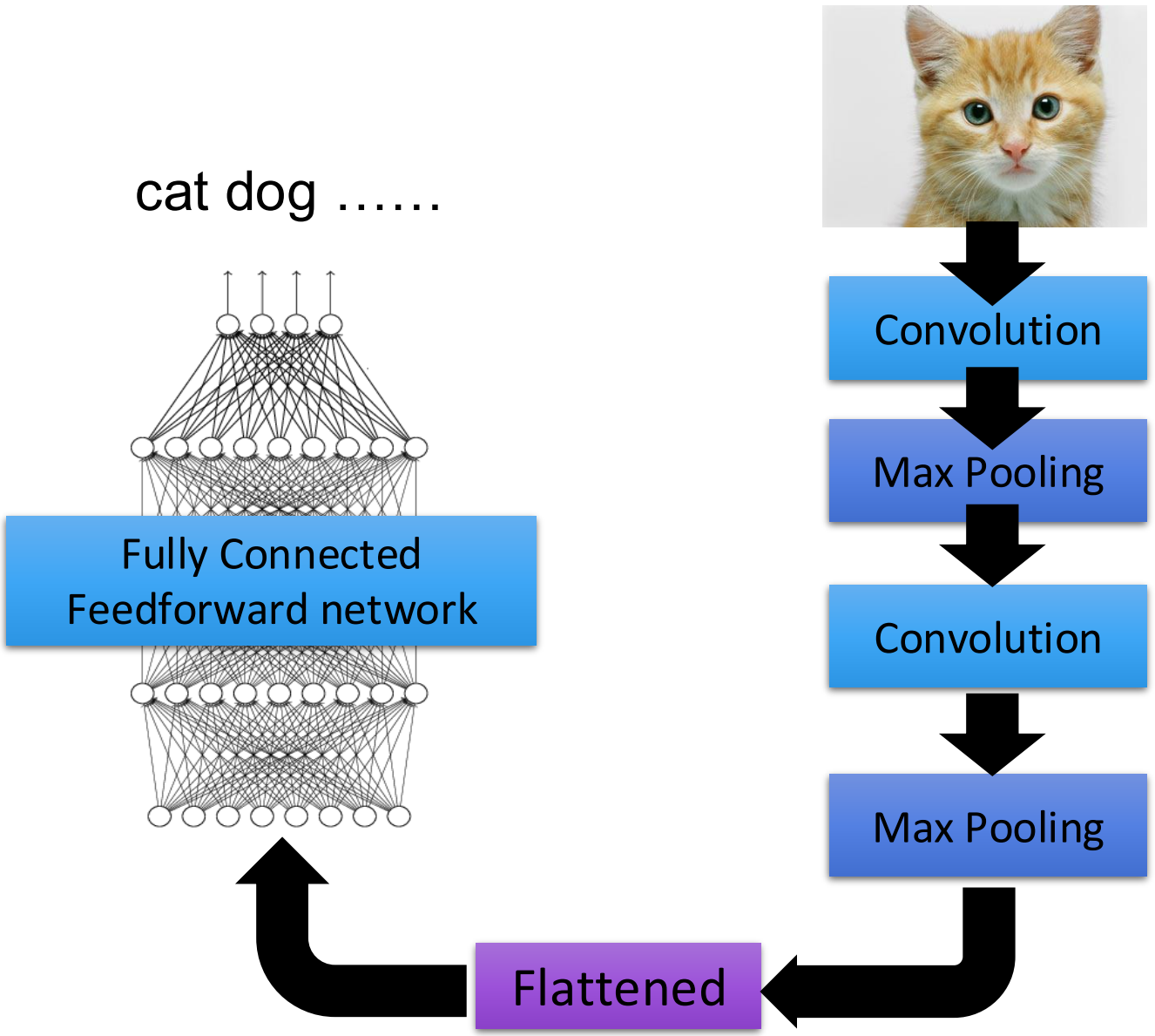



cat dog ……
Fully Connected
Feedforward network
Convolution
Max Pooling
Convolution
Max Pooling
Can repeat many times
Flattened


cat dog ……
Fully Connected
Feedforward network
Convolution
Max Pooling
Convolution
Max Pooling
Flattened

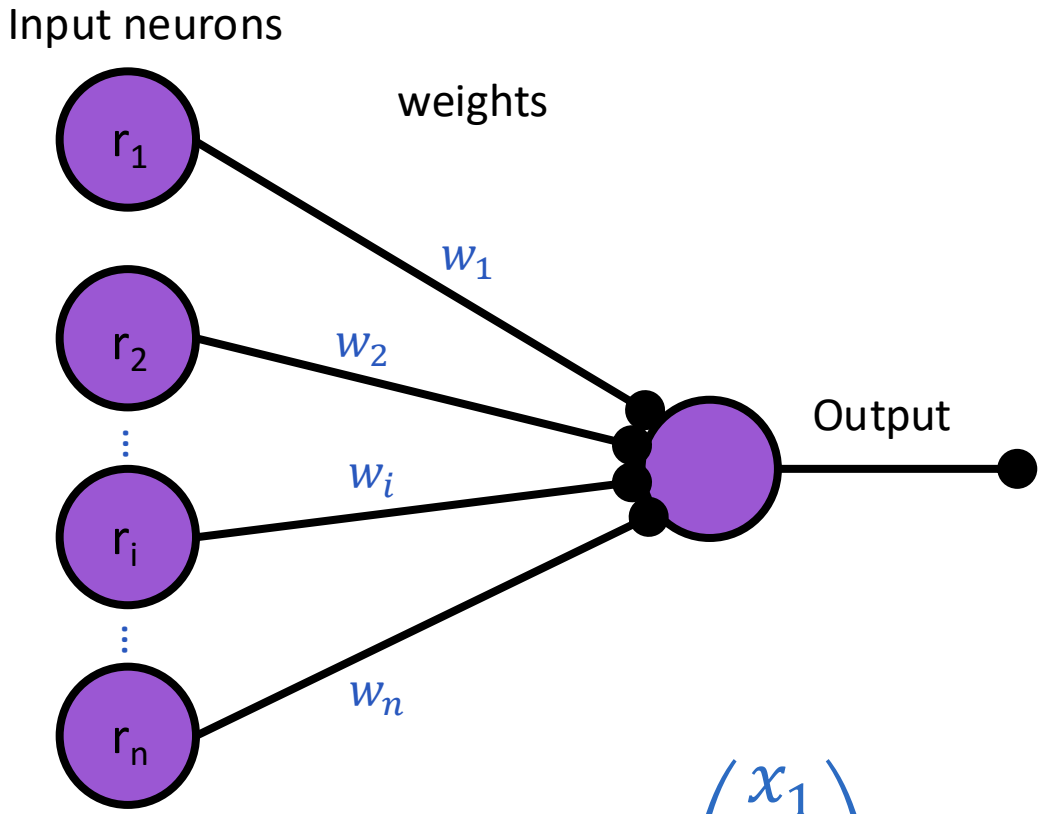
Flattening


-1
1
0
3
Flattened
3
0
1
3
-1
1
0
3
Fully Connected
Feedforward network


cat dog ……
Fully Connected
Feedforward network
Convolution
Max Pooling
Convolution
Max Pooling
Flattened

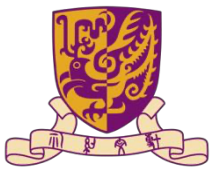
Recall: Linear Algebra

- The vector x is the input vector, from the CNN
- The weight w is the classifier



$$(w_1 \quad w_2 \quad \cdots \quad w_N)\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{pmatrix}$$

$$= w_1x_1 + w_2x_2 + \cdots + w_Nx_N$$



Activation Functions

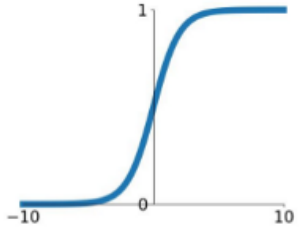
Final output of each neuron: $a(w_1x_1 + w_2x_2 + \cdots + w_Nx_N)$

where $a(.)$ is an activation function:

Activation Functions

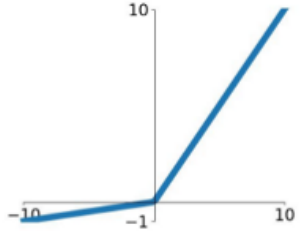
Sigmoid

$\sigma(x) = \frac{1}{1+e^{-x}}$



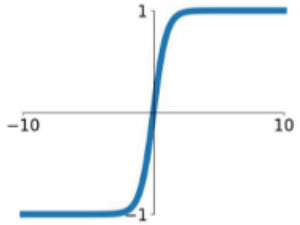
Leaky ReLU

$\max(0.1x, x)$



tanh

$\tanh(x)$

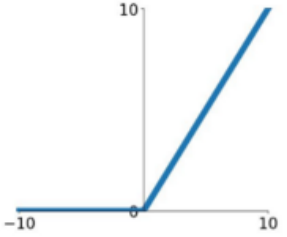


Maxout

$\max(w_1^T x + b_1, w_2^T x + b_2)$

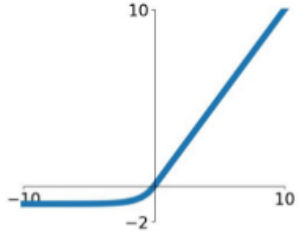
ReLU

$\max(0, x)$



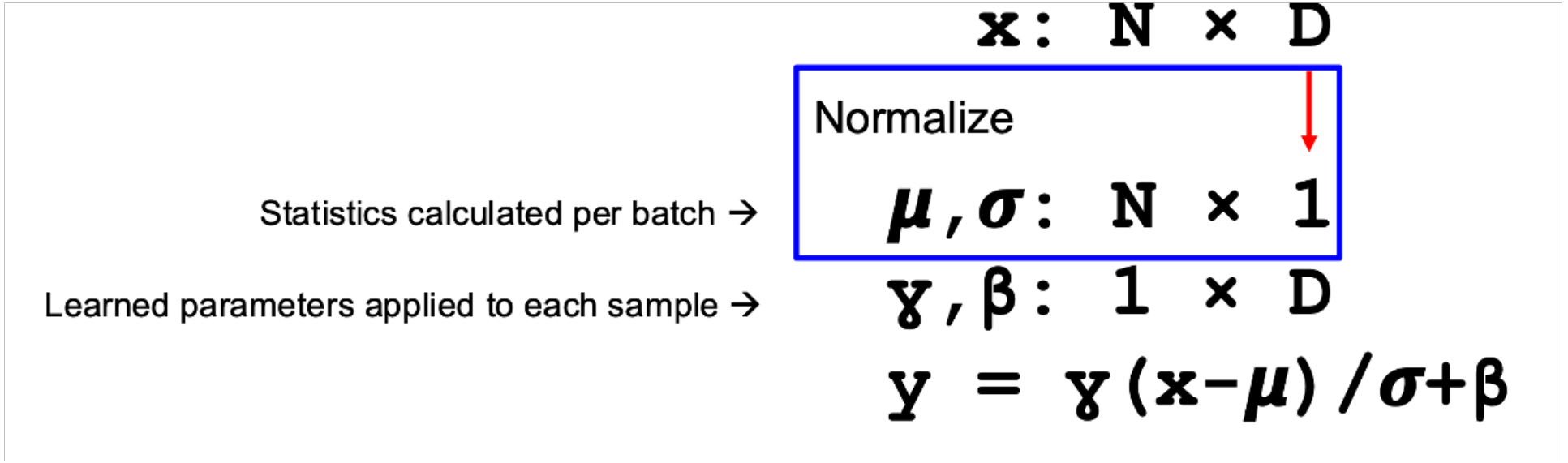
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



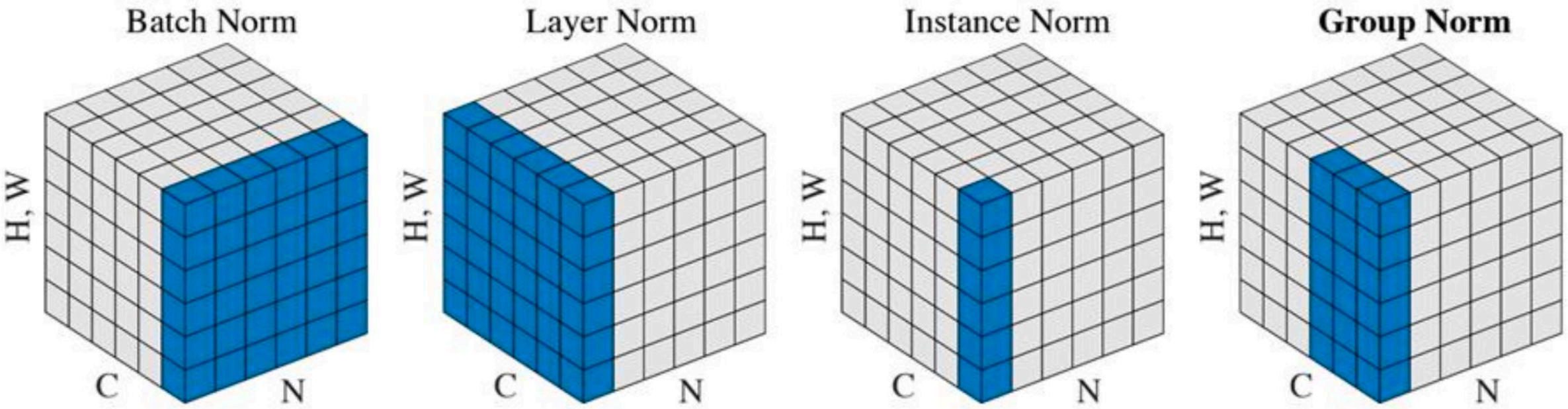
Example Normalization Layer: LayerNorm

- High-level Idea: Learn parameters that let us scale / shift the input data
  - Normalize input data
  - Scale / shift using learned parameters





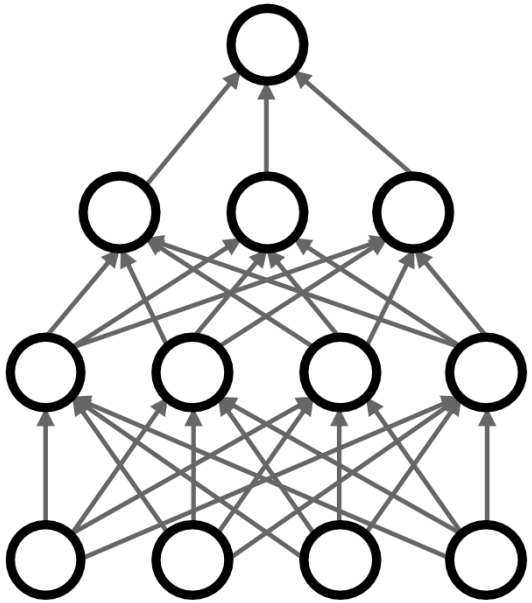
Other Normalization Layers

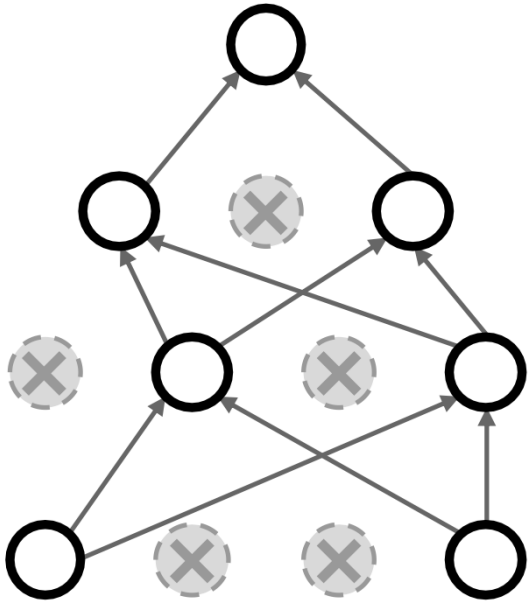




Regularization: Dropout

- In each forward pass, randomly set some neurons to zero
- Probability of dropping is a hyperparameter; 0.5 is common

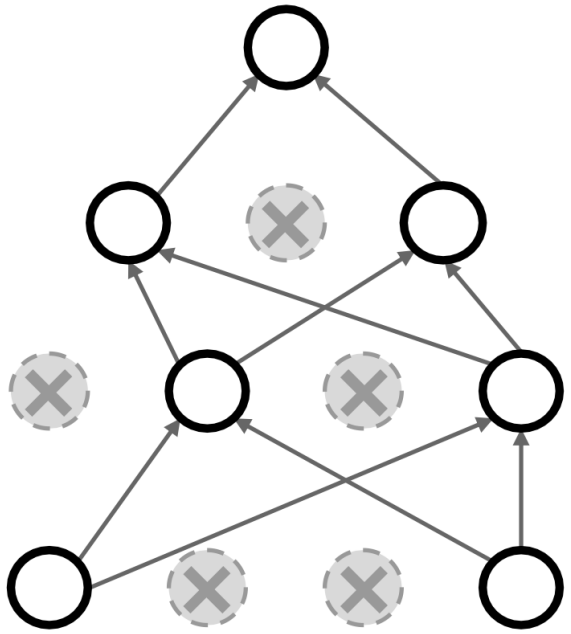




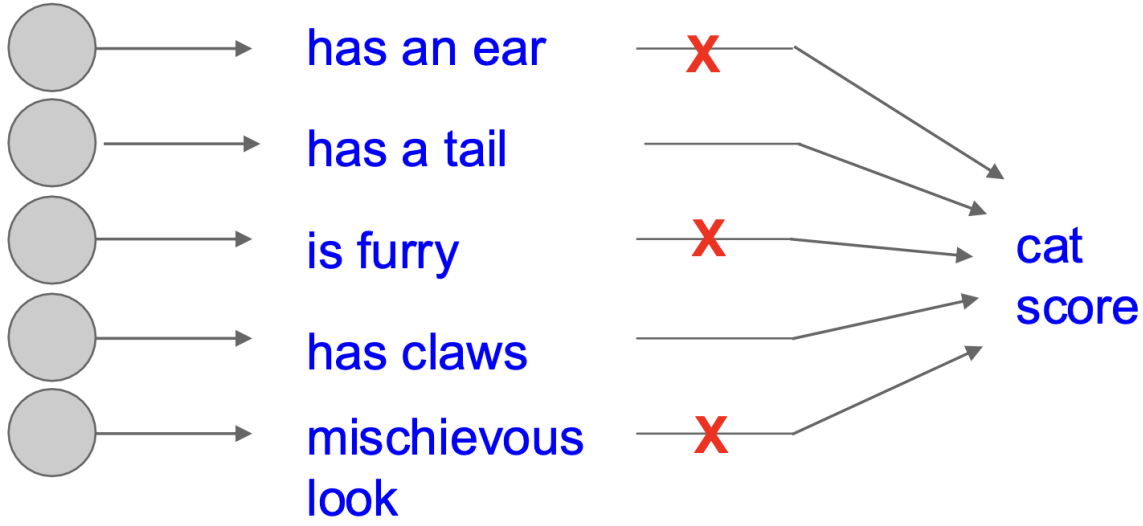


Regularization: Dropout

- How can this possibly be a good idea?



Forces the network to have a redundant representation;
Prevents co-adaptation of features





Summary of Convolutional Layer

- Accepts a volume of size $W_1 \times H_1 \times D_1$, requires four hyperparameters:
  - Number of filters K
  - the stride S
  - the amount of zero padding P.
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:

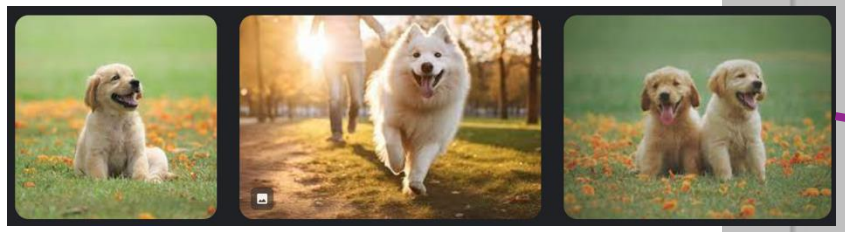
$$W_2 = (W_1 + 2P)/S + 1$$

$$H_2 = (H_1 + 2P)/S + 1$$

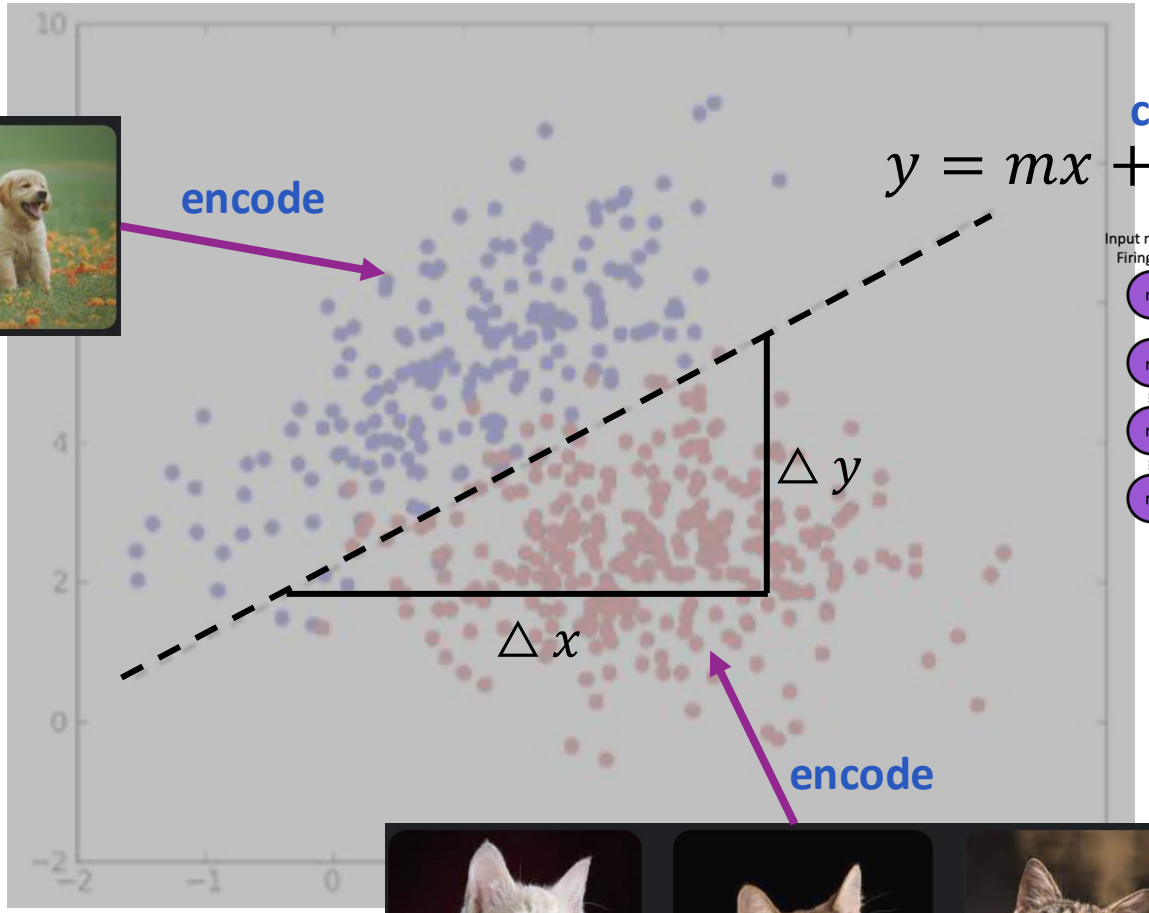
$$D_2 = K$$

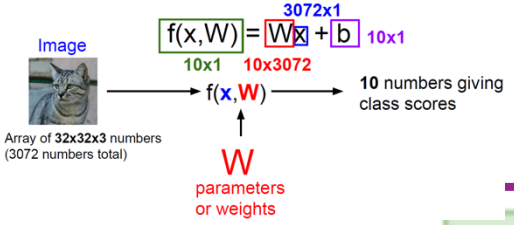
Need to make sure that (W1 + 2P)/S and (W2 + 2P)/S are integers

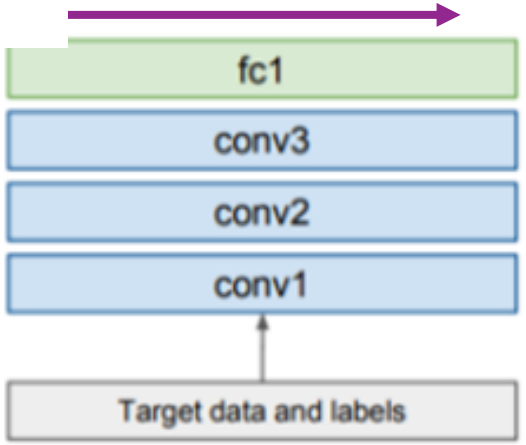




encode

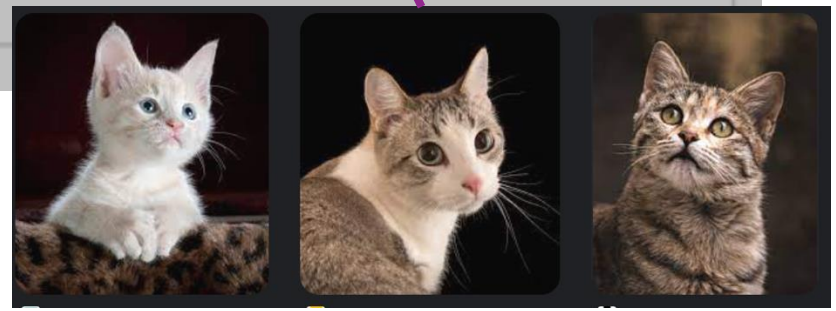






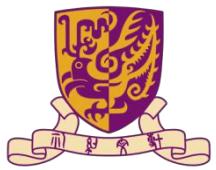
Pre-train model (encoder)

encode





CNN in Pytorch





python Copy

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(32 * 8 * 8, 128)
        self.fc2 = nn.Linear(128, 10)  # 10 classes for example

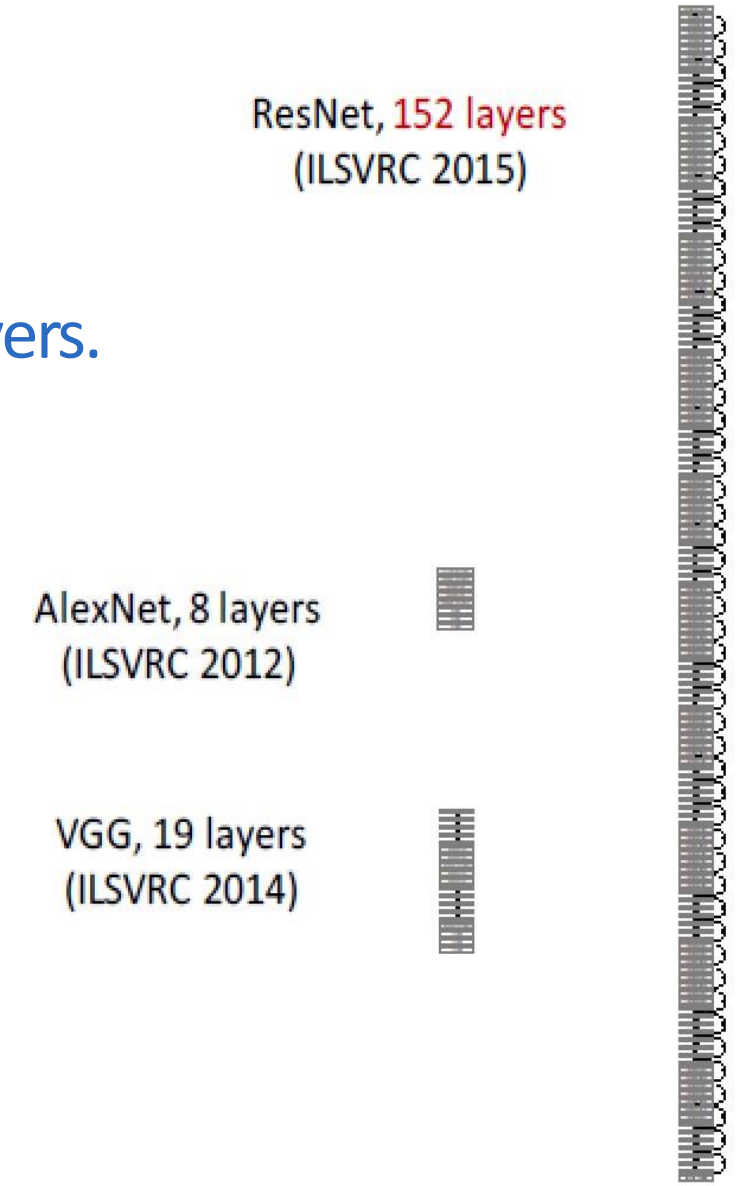
    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.max_pool2d(x, 2, 2)
        x = F.relu(self.conv2(x))
        x = F.max_pool2d(x, 2, 2)
        x = x.view(-1, 32 * 8 * 8)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

# Initialize the network
cnn = CNN()

# Define the loss function and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(cnn.parameters(), lr=0.001)
```

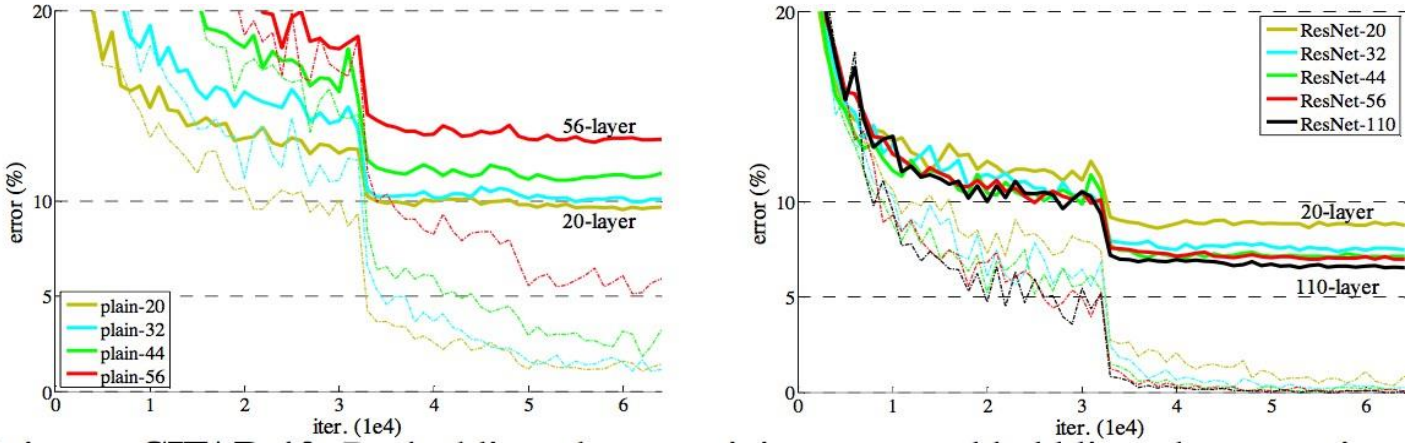
Deep Residual Networks (ResNet)

- Winner of ILSVRC 2015, beating others by large margins.
- It goes much deeper than previous methods with 152 layers.
- It brings about the revolution of depth.



Deep Residual Networks (ResNet)

- Top-5 error on Image Net.
  - 152-layer ResNet: 4.49%
  - GoogLeNet: 6.6%
  - Humans: 5.1%



- ResNet classification performance leveled off by 150.

- What are all these layers doing? We don't have a clear answer, but the idea that they're computing increasingly abstract features is starting to sound fishy...

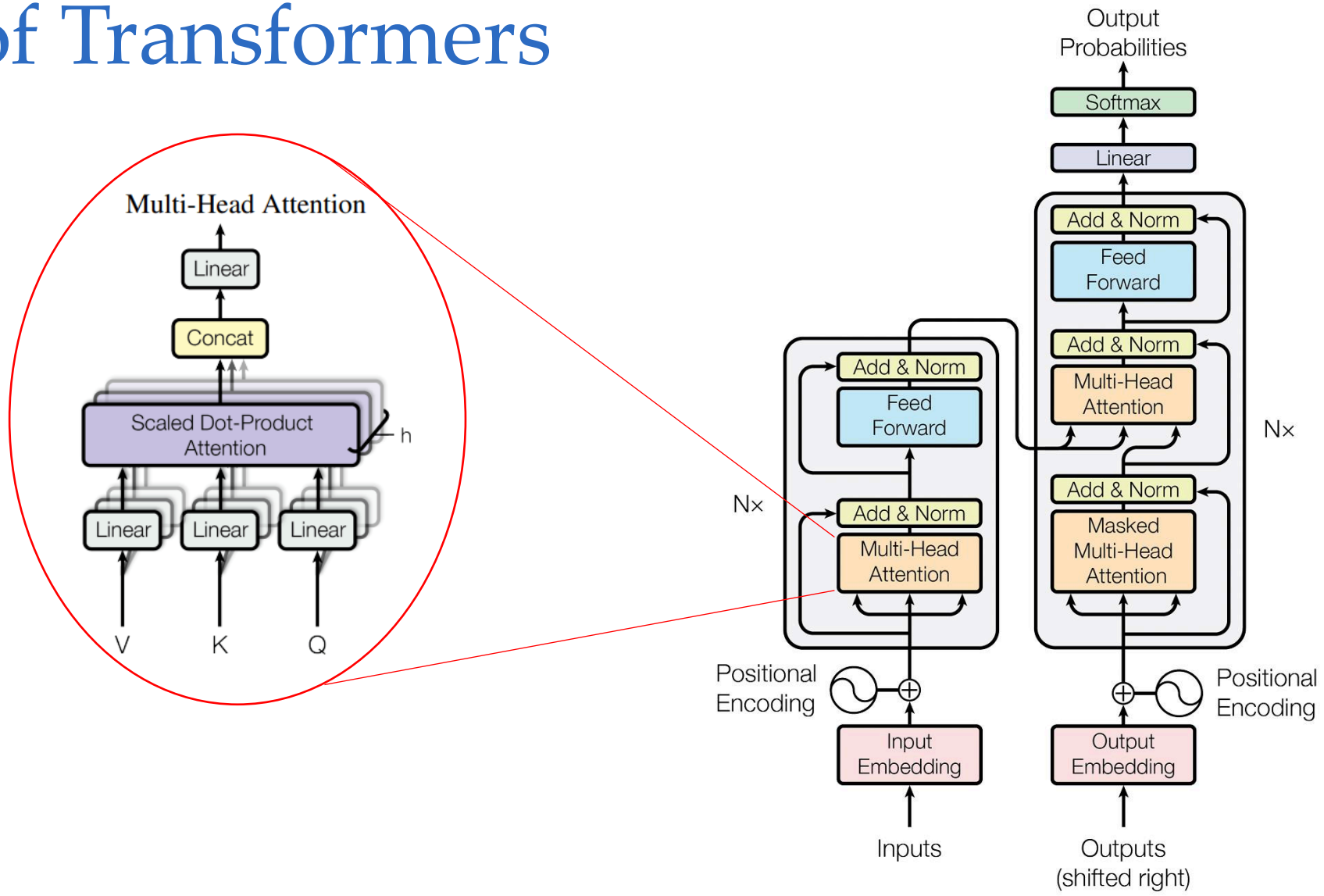


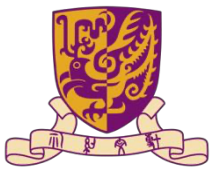
Agenda

- Introduction to convolutional layer
- Convolutional Neural Networks (CNN)
- Visual Transformer



Review of Transformers

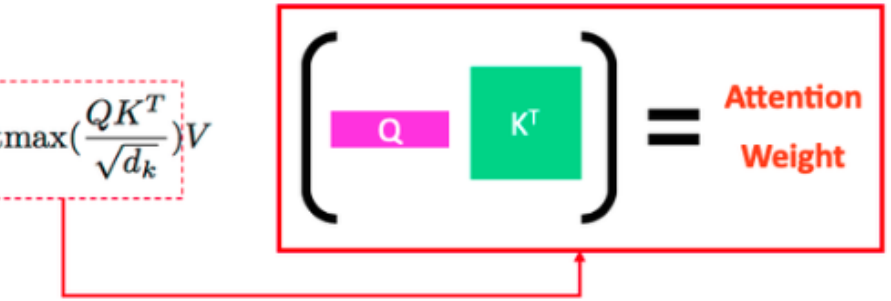


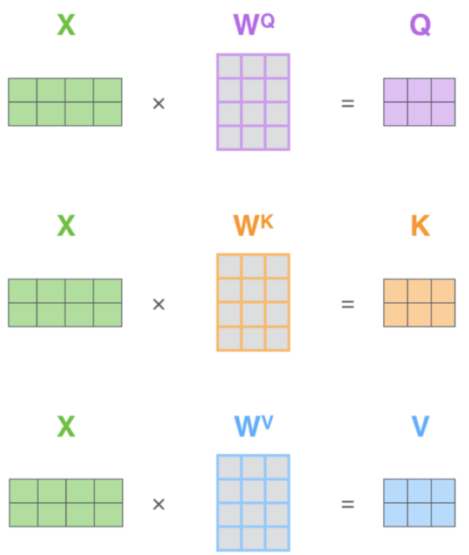


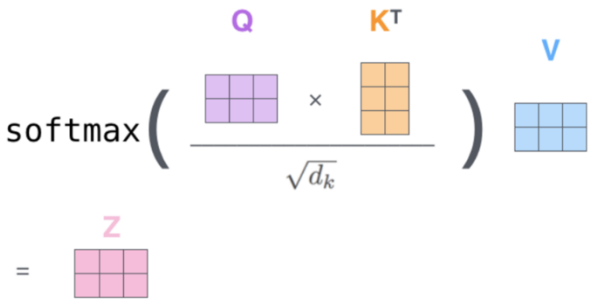
Recall: Details of Attention Layers

- The Q, K and V are formed using **the same input X** with different weight, and a softmax is used to compute **how much attention is put on V**.
- All the weight can be trained using some targets
- It is followed by a feed forward network

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

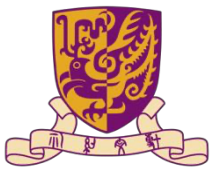






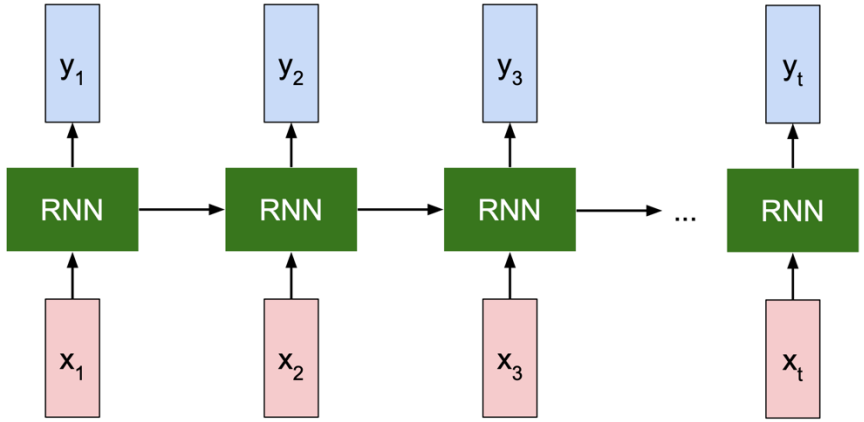
Recall: Self-attention Laver

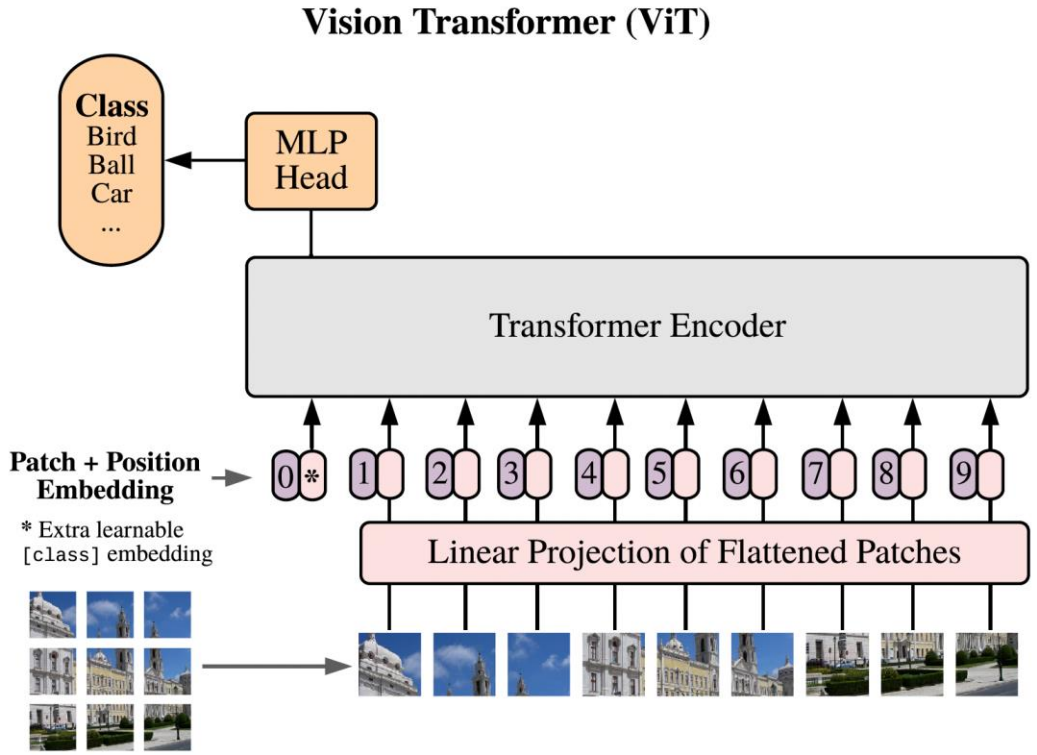
Word	Value vector	Score	Value X Score
<s>		0.001	
a		0.3	
robot		0.5	
must		0.002	
obey		0.001	
the		0.0003	
orders		0.005	
given		0.002	
it		0.19	
		Sum:	

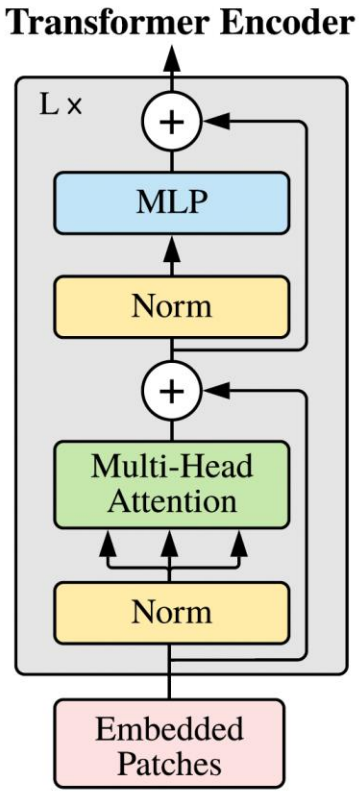


Transformers in Computer Vision

- Fundamental challenge: How to create a sequence from an image?

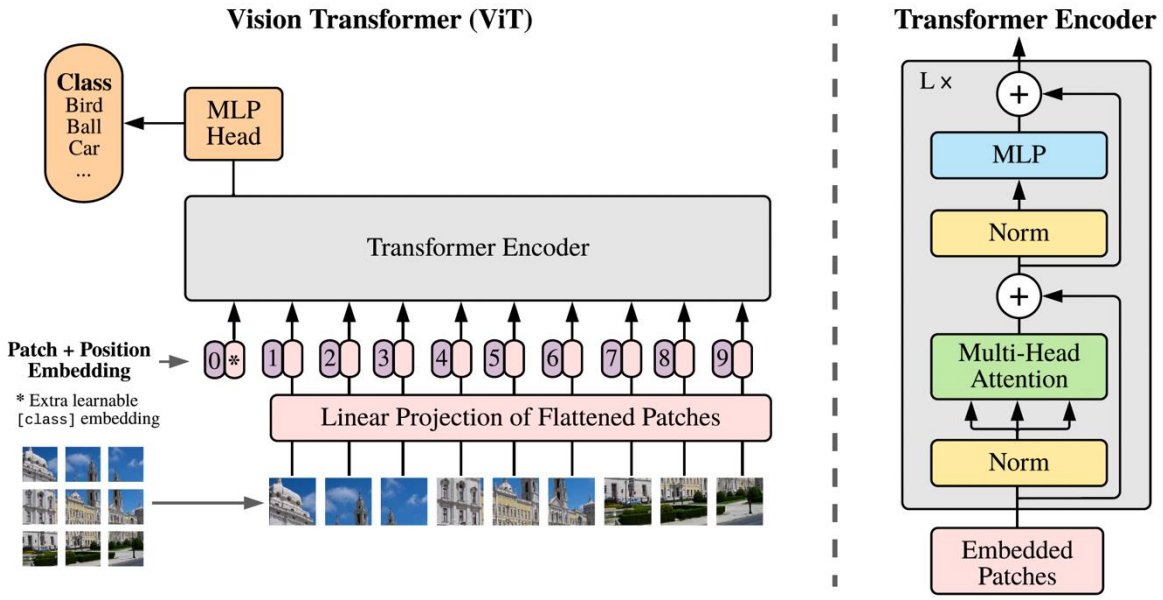






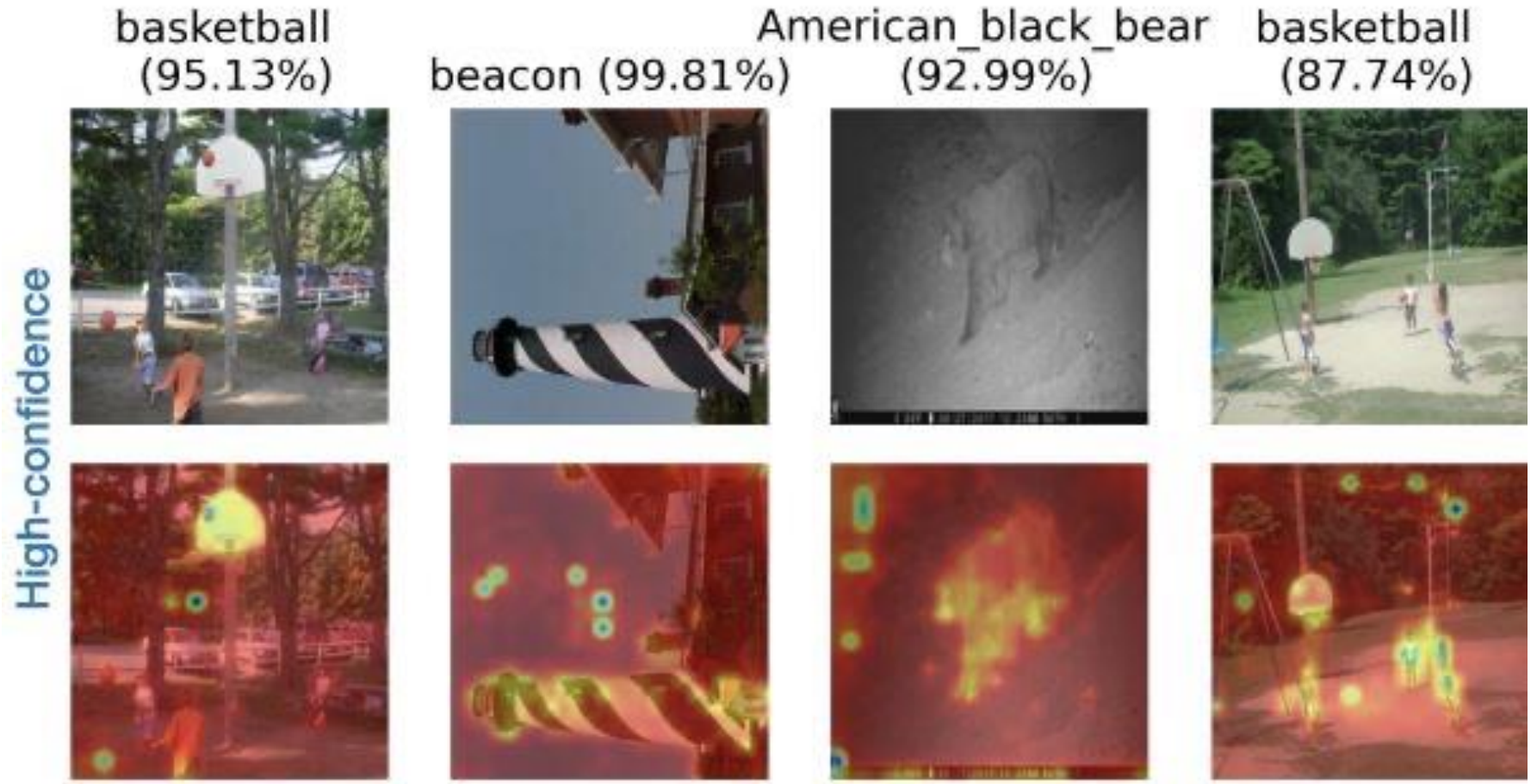
Visual Transformer: ViT Architecture

- Using image patch + position embeddings as the tokens
- Split an image into patches, feed linearly projected patches into standard transformer encoder
- With patches of 14x14 pixels, you need 16x16=256 patches to represent 224x224 images

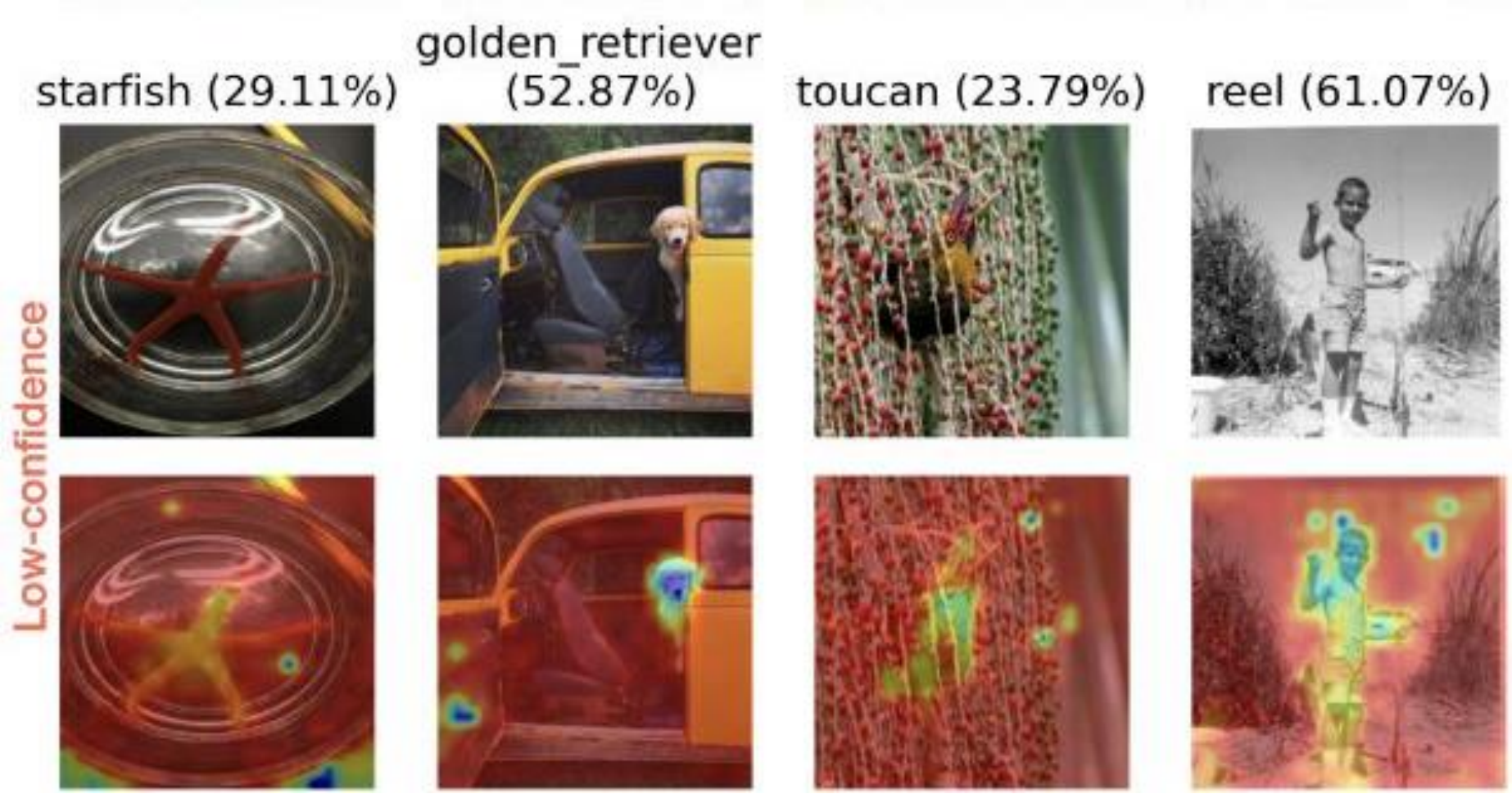


ref: AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

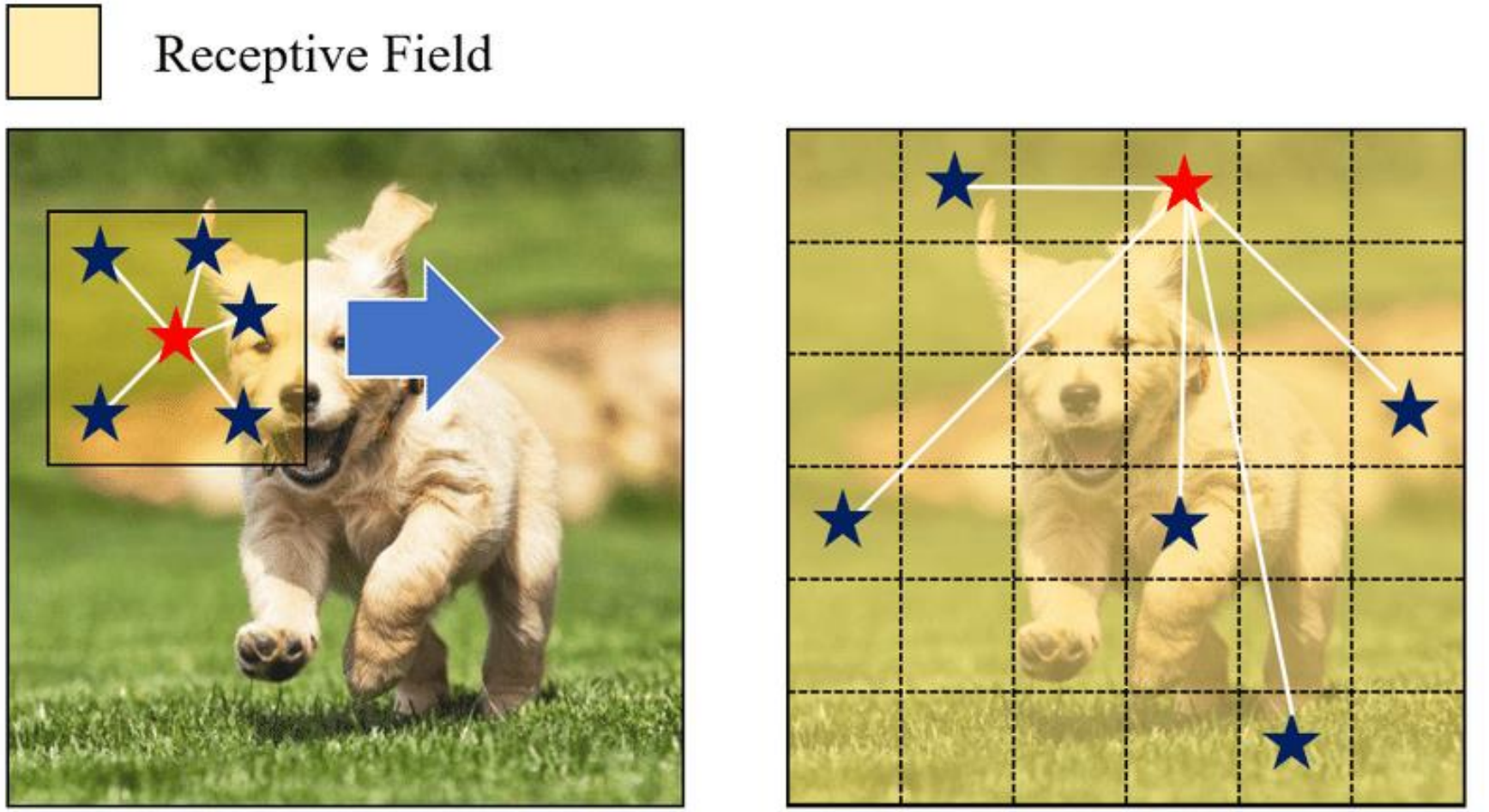
Attention Maps



Attention Maps



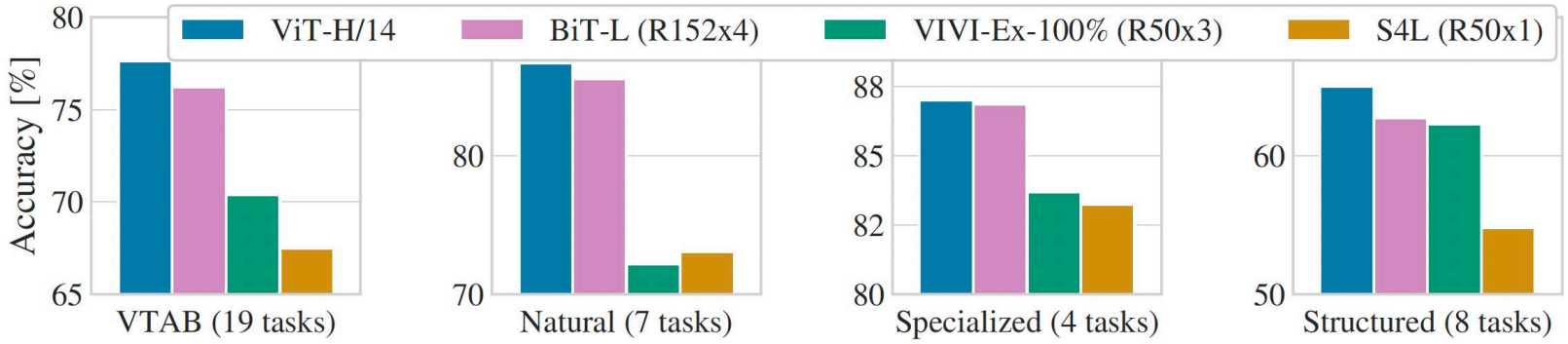
ViT vs. CNNs


Receptive Field
Convolution of CNN
Attention of Vision Transformer

ViT outperforms CNN-based model

- JFT: contains 300M data for training, not publicly available
- Faster than CNN

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	**88.55** ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	**90.72** ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	**99.50** ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	–
CIFAR-100	**94.55** ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	–
Oxford-IIIT Pets	**97.56** ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	–
Oxford Flowers-102	99.68 ± 0.02	**99.74** ± 0.00	99.61 ± 0.02	99.63 ± 0.03	–
VTAB (19 tasks)	**77.63** ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	–
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k



Vision transformer (ViT)

- Transformers are claimed to be more computationally efficient than CNNs or hybrid architectures

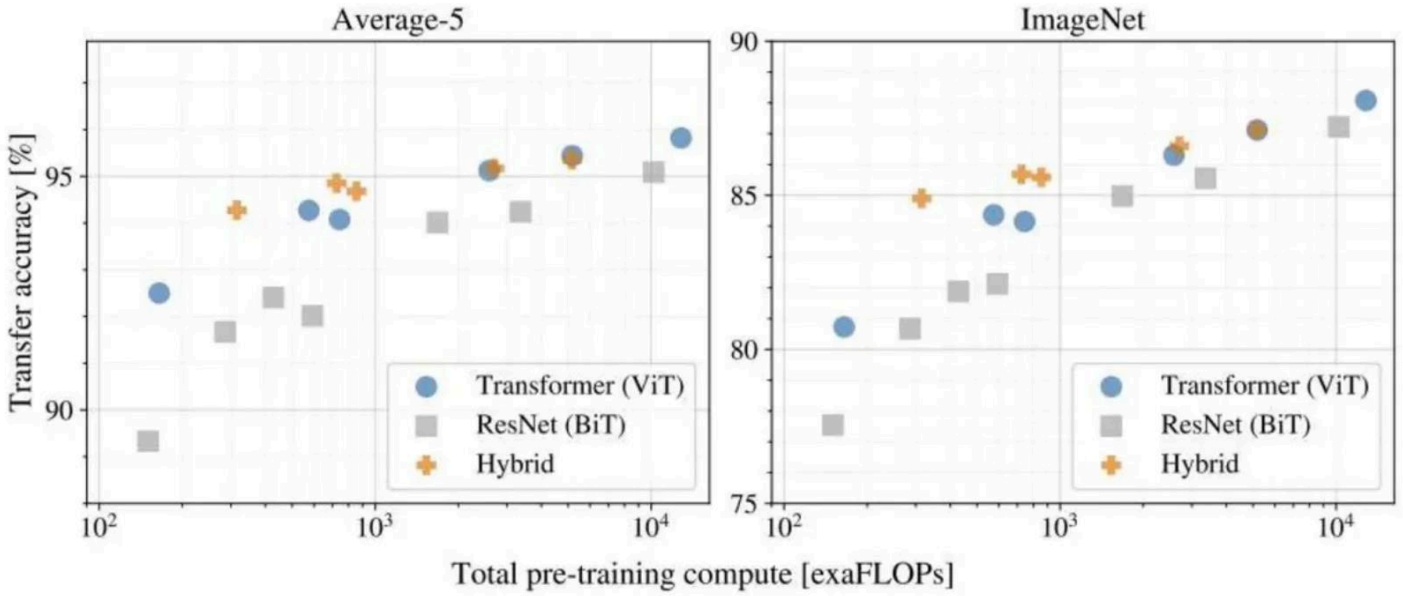


Figure 5: Performance versus cost for different architectures: Vision Transformers, ResNets, and hybrids. Vision Transformers generally outperform ResNets with the same computational budget. Hybrids improve upon pure Transformers for smaller model sizes, but the gap vanishes for larger models.



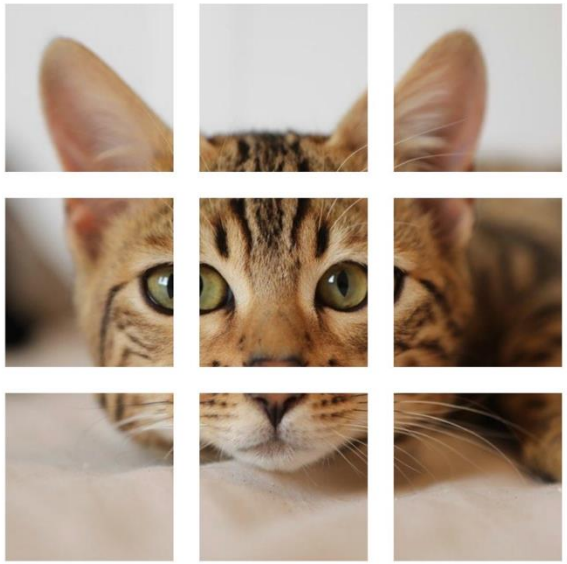
Challenges of Vision Transformers

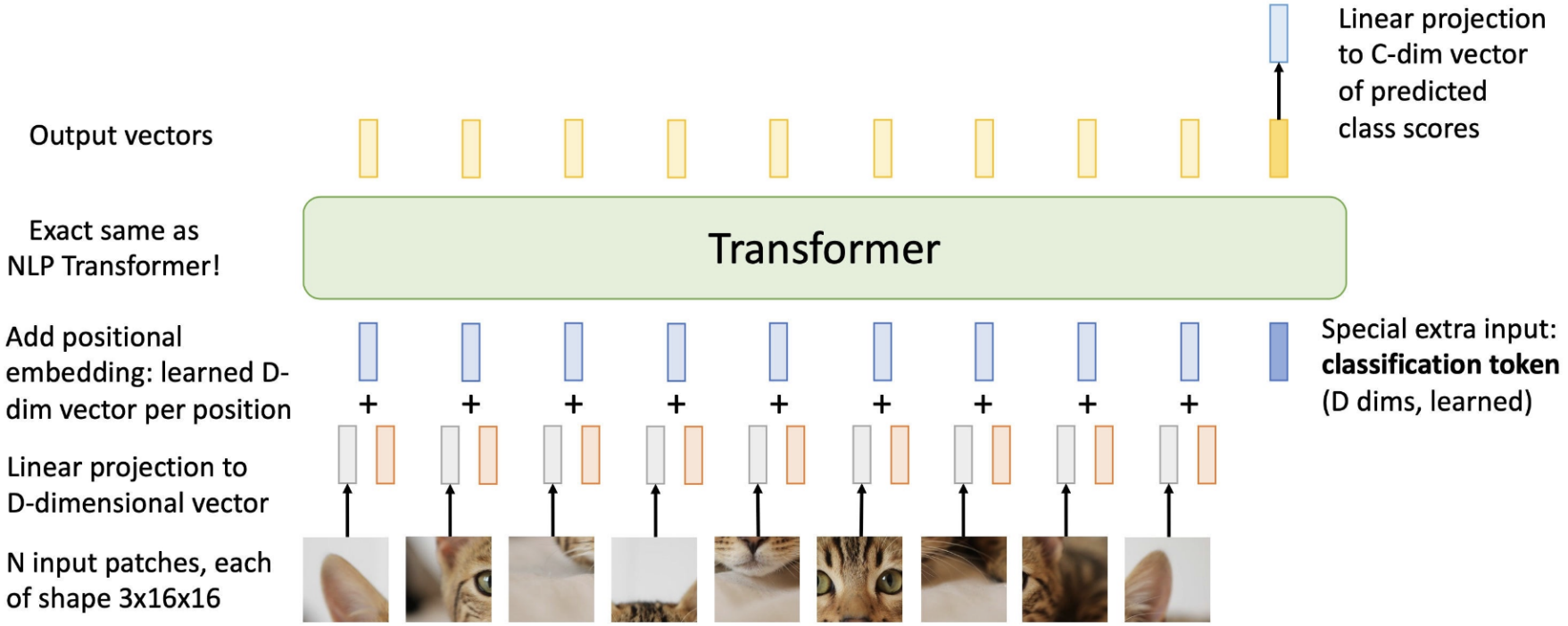
- rely heavily on massive amounts of data for large-scale training. This is why the quality of data significantly influences the generalization and robustness of transformers in computer vision tasks.

- Whilst ViT shows exceptional performance on downstream image classification tasks, directly applying the ViT backbone on object detection has failed to surpass the results of CNNs



Vision Transformer as Image Encoder Architecture

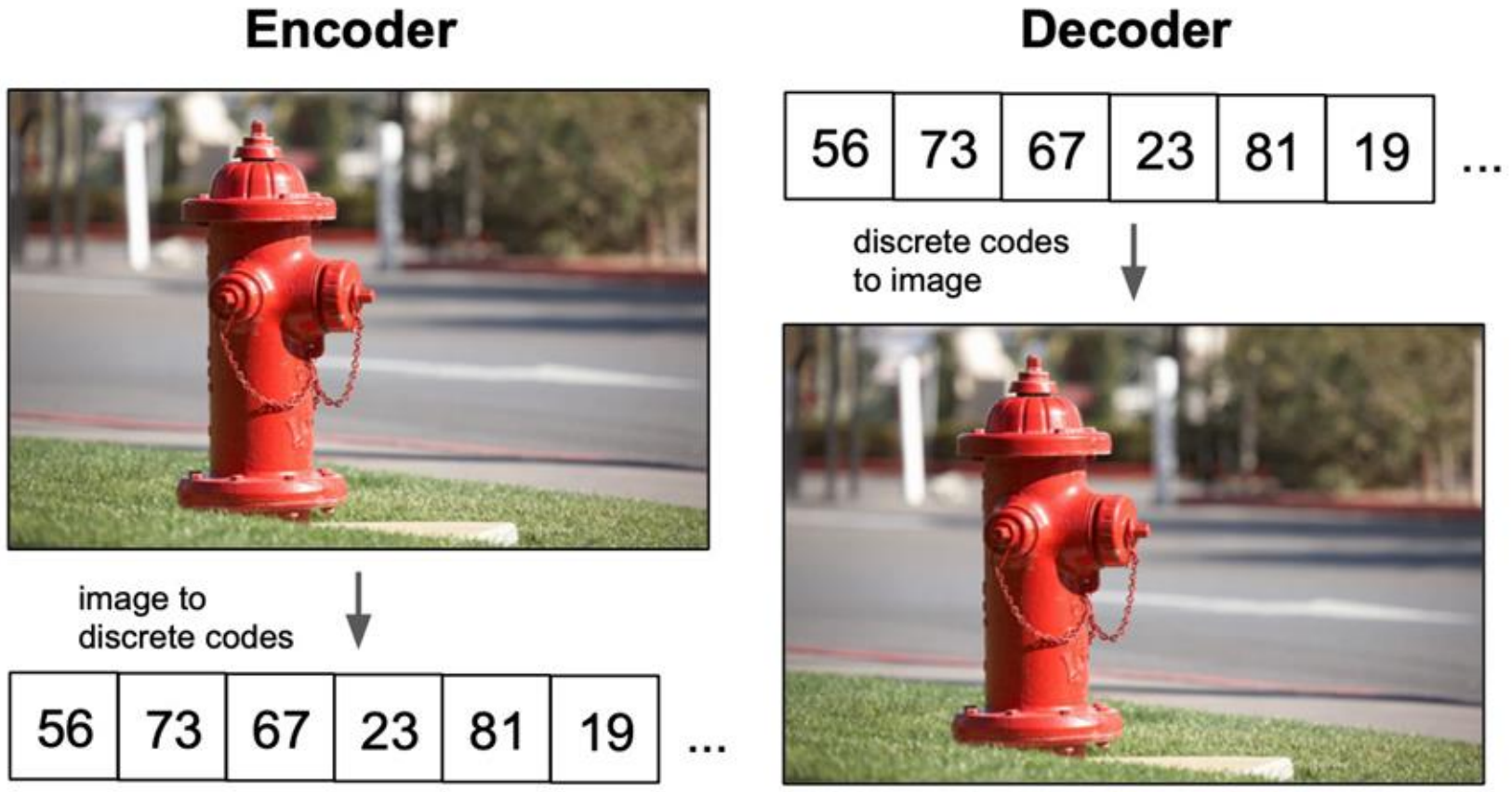






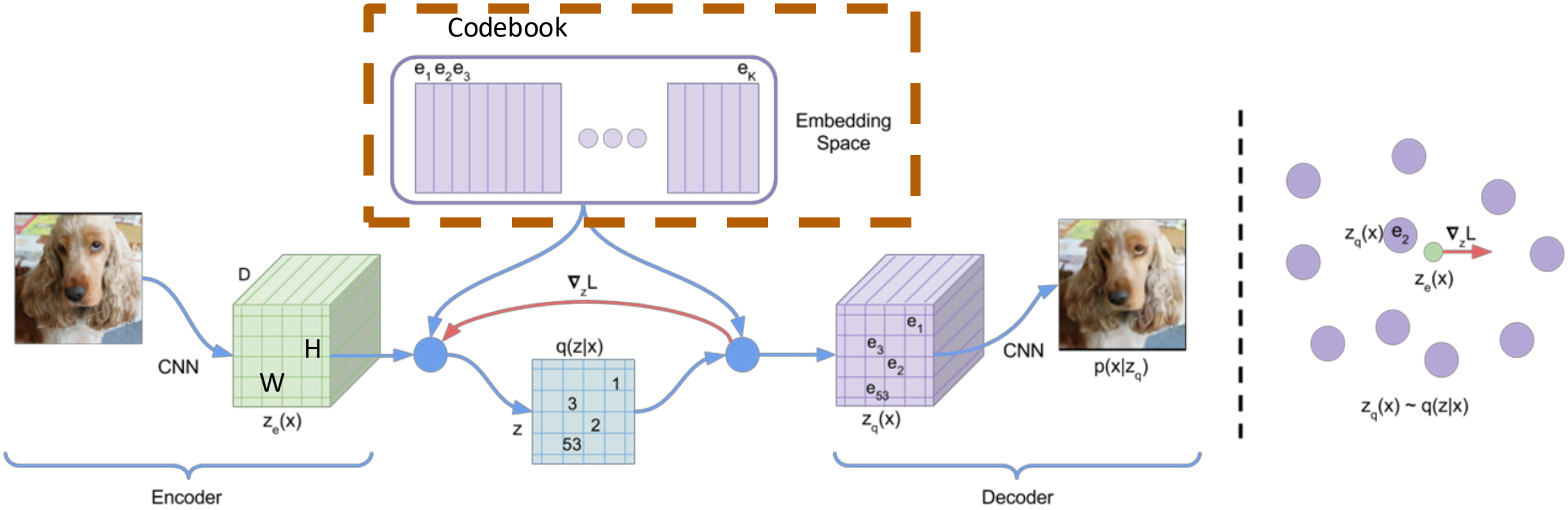
Recall: Vector Quantized-VAE (VQVAE)

- VAE: learns a continuous latent representation
- Vector Quantized-VAE: learns a discrete latent representation



ref: Neural Discrete Representation Learning

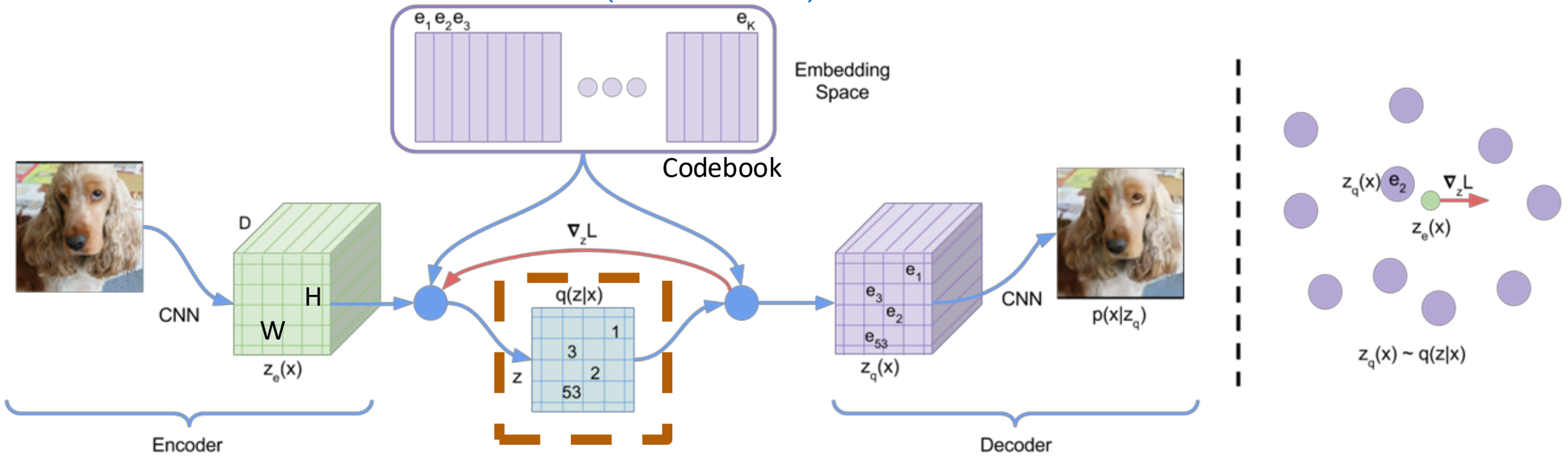
Vector Quantized-VAE (VQVAE)



- Codebook: list of K vectors with dimension D associated with index 1,2,...K
- Latent embedding vector: $e_i \in R^D$, i ∈ 1, 2, …, K



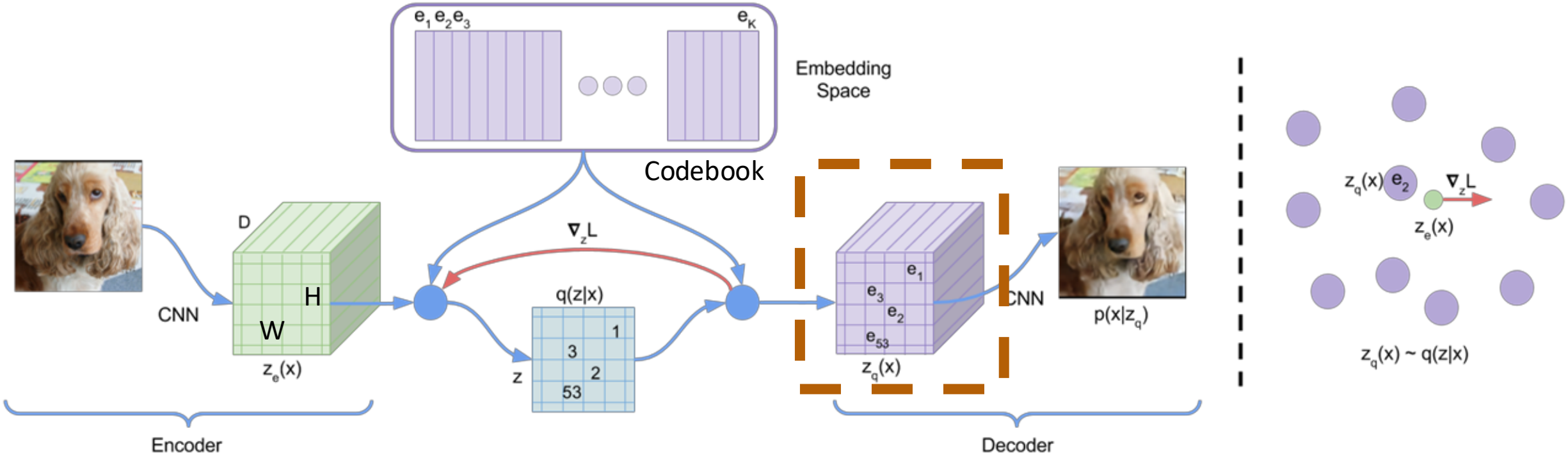
Vector Quantized-VAE (VQVAE)



- Discrete latent variables z are calculated by a nearest neighbour look-up using the shared embedding space (codebook)
- Posterior distribution ***q(z|x)*** probabilities defined as one hot:

$$q(z=k|x) = \begin{cases} 1 & \text{for } k = \text{argmin}_j \|z_e(x) - e_j\|_2, \\ 0 & \text{otherwise} \end{cases}$$

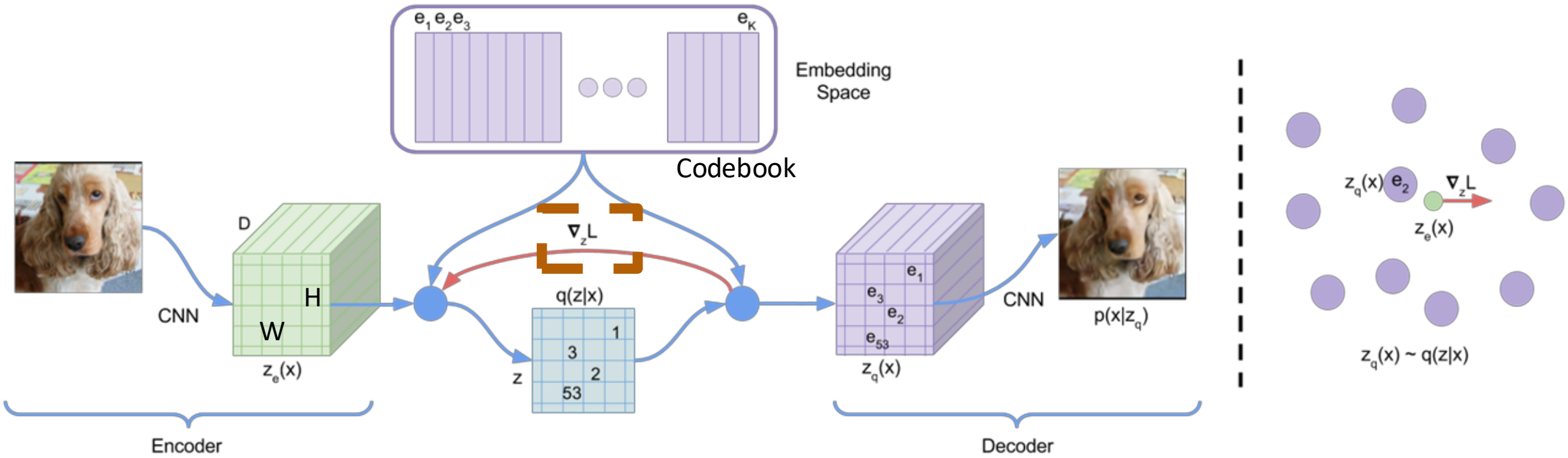
Vector Quantized-VAE (VQVAE)



The input to the decoder is the corresponding embedding vector e_k:

$$z_q(x) = e_k, \quad \text{where} \quad k = \text{argmin}_j \|z_e(x) - e_j\|_2$$

Vector Quantized-VAE (VQVAE)



- Copy gradients $\nabla_z L$ from decoder input $z_q(x)$ to encoder output $z_e(x)$
- Train the network with loss:

sg: stop-gradient operator

$$L = \underbrace{\log p(x|z_q(x))}_{\text{decoder}} + \underbrace{\|\text{sg}[z_e(x)] - e\|_2^2}_{\text{codebook}} + \underbrace{\beta\|z_e(x) - \text{sg}[e]\|_2^2}_{\text{encoder}}$$

python Copy

```python
from transformers import ViTModel, ViTImageProcessor
import torch
from PIL import Image
import requests

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# Load the ViT model for obtaining embeddings
model = ViTModel.from_pretrained('google/vit-base-patch16-224')
model.to(device)

processor = ViTImageProcessor.from_pretrained('google/vit-base-patch16-224')

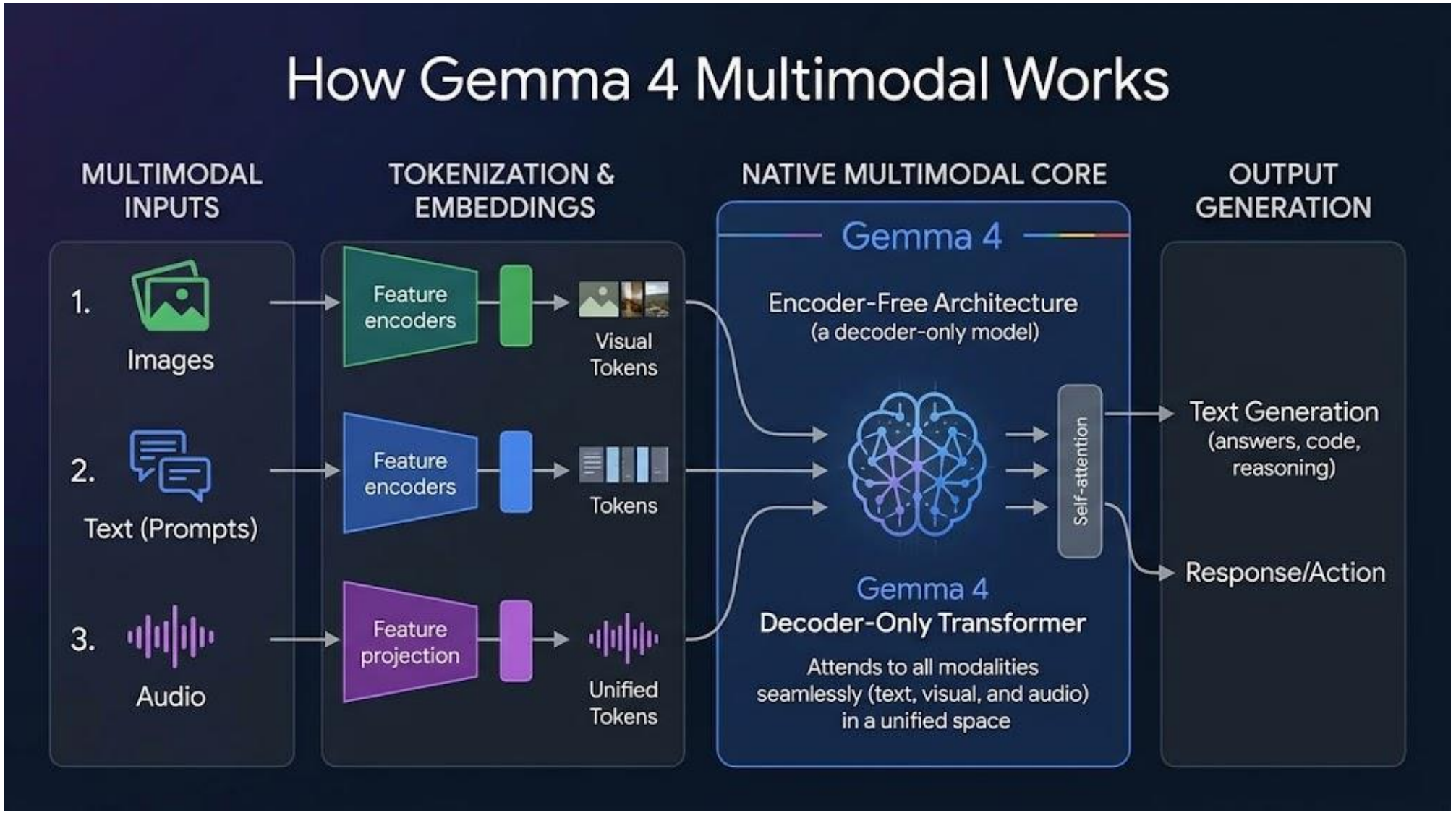
# Load the image from the URL
url = 'https://images.theconversation.com/files/540464/original/file-20230801-23-a5tyah.jpg'
image = Image.open(requests.get(url, stream=True).raw)

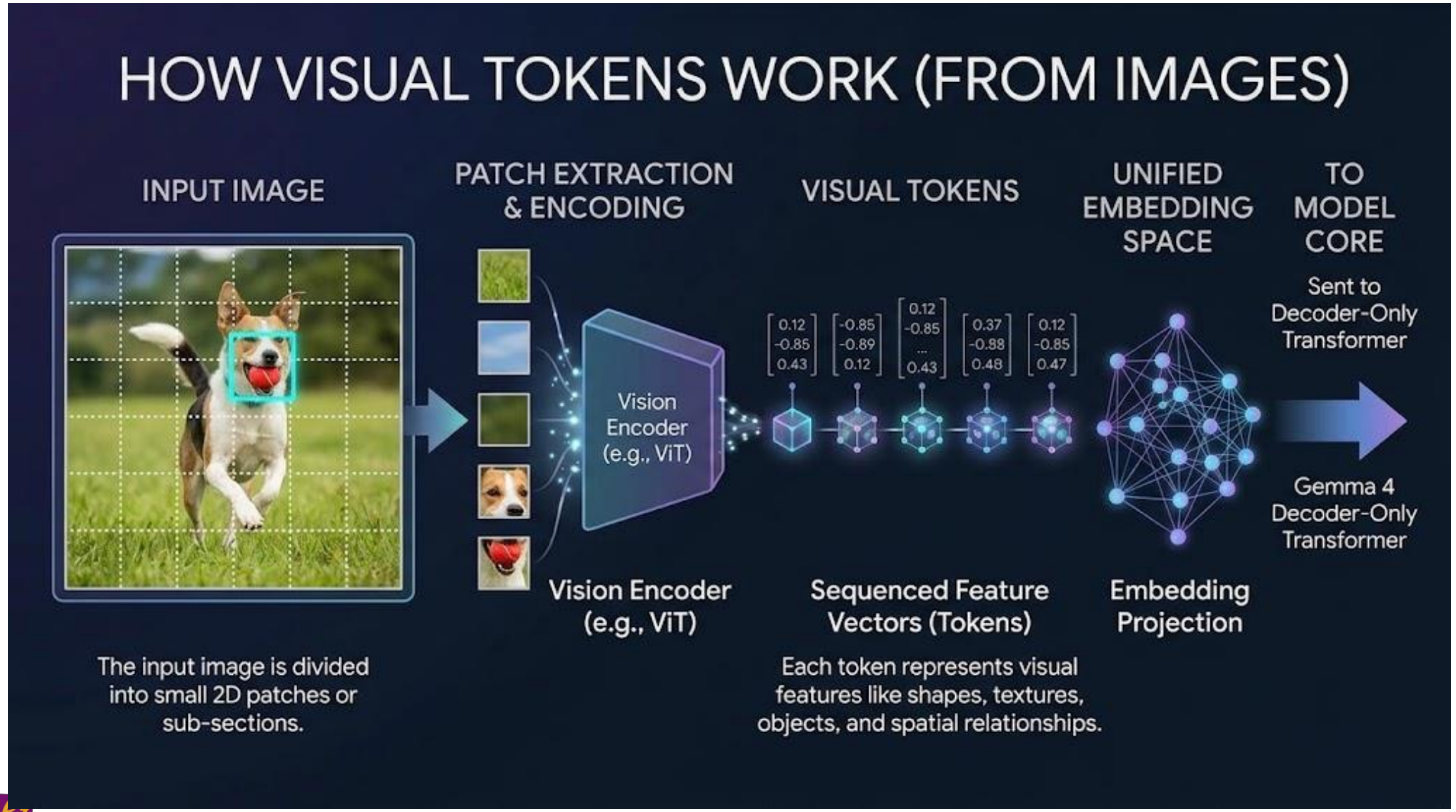
# Process the image
inputs = processor(images=image, return_tensors="pt").to(device)

# Get the embeddings
with torch.no_grad():
    outputs = model(inputs.pixel_values)
    embeddings = outputs.last_hidden_state[:, 0, :]  # Get the embedding for the [CLS] token

# Print the shape of the embeddings
print("Shape of the embeddings:", embeddings.shape)
```




How Gemma 4 Multimodal Works
MULTIMODAL INPUTS
TOKENIZATION & EMBEDDINGS
NATIVE MULTIMODAL CORE
OUTPUT GENERATION
1.
Images
2.
Text (Prompts)
3.
Audio
Feature encoders
Feature encoders
Feature projection
Visual Tokens
Tokens
Unified Tokens
Gemma 4
Encoder-Free Architecture
(a decoder-only model)
Self-attention
Gemma 4
Decoder-Only Transformer
Attends to all modalities seamlessly (text, visual, and audio) in a unified space
Text Generation
(answers, code, reasoning)
Response/Action

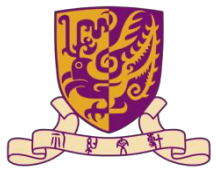

HOW VISUAL TOKENS WORK (FROM IMAGES)
INPUT IMAGE
PATCH EXTRACTION & ENCODING
VISUAL TOKENS
UNIFIED EMBEDDING SPACE
TO MODEL CORE
Vision Encoder (e.g., ViT)
0.12 -0.85 0.43
-0.85 -0.89 0.12
0.12 -0.85 ... 0.43
0.37 -0.88 0.48
0.12 -0.85 0.47
Sent to Decoder-Only Transformer
Gemma 4 Decoder-Only Transformer
Vision Encoder (e.g., ViT)
Sequenced Feature Vectors (Tokens)
Embedding Projection
The input image is divided into small 2D patches or sub-sections.
Each token represents visual features like shapes, textures, objects, and spatial relationships.

Primary Causes of the Crash of Air France Flight 447

- **Sensor Icing:** Ice clogged the pitot tubes, causing inaccurate airspeed readings and automatically disconnecting the autopilot.

- **Pilot Error:** Disoriented by the sudden failure, the pilot pulled back on the stick, pitching the nose up and forcing the plane into a stall.

- **Unrecovered Stall:** The crew failed to recognize the stall warnings and never pushed the nose down to regain airspeed before impact.



Medical

Clinical / Technical Domain	Specific Manifestation of Deskilling	Measurable Operational Consequence
Gastroenterology (Budzyń et al.)	Routine exposure to computer-aided polyp detection	20% relative drop in unassisted adenoma detection rate (p = 0.0089)
Breast Radiology (Controlled Multi-Reader Trials)	Over-reliance on erroneous automated prompts	Up to 12% increase in false-positive recalls among experienced readers
Digital Pathology (Multi-Center Diagnostic Trials)	Exposure to incorrect algorithm suggestions under time pressure	>30% of clinicians reversed initially correct human diagnoses
Commercial Aviation (Long-Haul Automation Analysis)	Chronic flight-deck reliance on automated cruise systems	Atrophy of manual stick-and-rudder aerodynamic recovery reflexes

https://medium.com/predict/ai-deskilling-in-medicine-what-did-the-lancet-study-reveal-1ae5faaaaed2

Some words about Coding Agents

- **Think before you prompt:** Engage with the problem and form your own hypothesis or solution before reaching for an AI tool.
- **Challenge the reasoning:** Ask the tool to explain *how* and *why* it reached its conclusion rather than passively accepting the output.
- **Accelerate, don't substitute:** Use AI to speed up tasks you already comprehend—never to bypass the process of understanding them.



Dimensions in Agentic Coding

- Architectural & Development Strategy
- Context & Knowledge Management
- Autonomy, Observability & Guardrails
- Human-in-the-Loop Workflow
- Evaluation, Verification & Economics
- Issues and Actions



Architectural & Development Strategy

- Align Before Writing Code & Structure Repos
  - Establish a shared mental model before generation to serve as explicit prompt assets.
  - Study agent capabilities, modules, and structure.
  - Restrict access so agents do not see all repositories, preventing confusion, hallucination, and codebase leaks.
  - Quick PoC to understand the approaches
- Build Deep, Not Wide
  - Design deep modules with simple, clear interfaces so AI models can navigate codebase dependencies effectively.
- Separate Brain from Hands into two distinct layers:
  - Ensures bugs are either reasoning choices (model) or runtime execution bugs (code).
  - The Brain (Model): Decides intent, plan, and tool selection.
  - The Harness (Code): Executes API calls, handles retries, parses responses, and enforces hard security and execution limits. Pick the right one for your task.
- Adopt Test-Driven Development (TDD): Implement red-green-refactor cycles to ship small, manageable changes that keep AI outputs verified at every step.



Context & Knowledge Management

- Use Modular Context & Hierarchy
  1. Prompt Engineering: Fast, cheap, reversible.
  2. Skills: scripts and domain knowledge into modular, on-demand reference directories
  3. Retrieval (RAG): Add dynamic/missing knowledge
  4. Fine-Tuning: Last resort for specialized behavior when large, clean datasets exist
- Maintain an Agent-Friendly & Version-Controlled Codebase
  - Remove obsolete warnings, bad documentation, and ambiguous references.
  - Store prompts/skills in version control (Git) alongside source code, paired with clear changelogs and verified against evaluation sets before merging.



Autonomy, Observability & Guardrails

- Put Hard Leashes on Autonomy
  - To prevent execution loops, cascading error compounding, and unmitigated costs across multi-step chains, enforce strict deterministic controls before deployment:
  - Hard step limits and max-iteration caps.
  - Financial spend/cost caps and API timeout limits.
  - Circuit breakers and automatic self-halting mechanisms.
- Log for Debugging
  - Implement structured, end-to-end tracing from day one. Capture every user input, retrieved document, tool execution, intermediate reasoning step, and timestamp to enable exact replay and debugging of silent failures.
- System-First Failure Investigation
  - When an agent fails, investigate system components before modifying prompts or upgrading models:
  - Check for stale, missing, or corrupted retrieved context.
  - Verify tool response payloads and silent API errors.
  - Check for missing retry logic or unhandled edge cases.

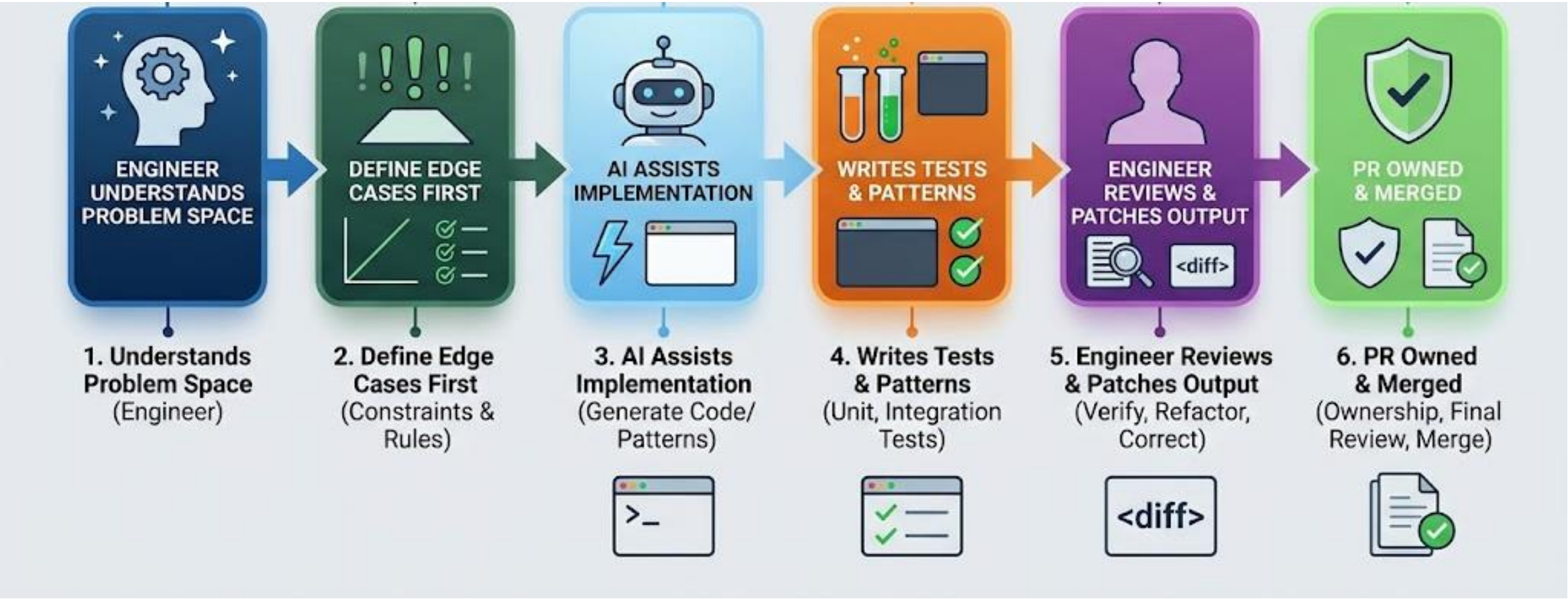


Autonomy, Observability & Guardrails

LAYER	WHEN	WHO	EVIDENCE	PROVES
L1 In-task loop	Repeatedly, in session	Agent self-check	Test output, build logs, screenshots	This change meets the spec
L2 Independent verifier	Once, after "done"	Fresh-context agent	Report checked against spec	The "done" claim is credible
L3 PR review	Before merge	AI (REVIEW.md) + human	Review comments + CI	No regressions, follows the rules
L4 Evals	On config change, on a schedule	CI reruns past tasks	Pass rate on historical tasks	The agent setup didn't regress
L5 Prod monitoring	Continuous after launch	Deterministic scripts	Error and failure rates	Real usage holds up

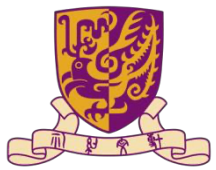


Human-in-the-Loop Workflow



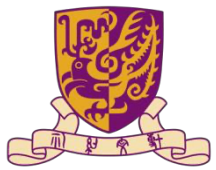
Human-in-the-Loop Workflow

- Define Edge Cases First
  - Identify domain requirements and edge cases prior to triggering boilerplate generation.
- Thorough Code Ownership & Patching
  - Validate structural and algorithmic trade-offs.
  - Patch domain-specific edge cases missed by the agent.
  - Rename variables to align precisely with the project's domain model.
  - Write **WHY** comments explaining architectural intent rather than simple **WHAT** comments describing syntax.



Evaluation, Verification & Economics

Principle	Execution	Purpose
Upfront Evals (Build First)	Define 20–30 real-world test cases with expected outputs before building features.	Replaces "vibe checks" with deterministic stopping criteria and regression testing.
Evaluation Packs	Implement self-verification automated test suites.	Prevents silent failures and speeds up review time.
Upfront Cost Modeling	Calculate price per task (token counts × call volume × max retries) prior to engineering.	Prevents architectural lock-in to economically unviable production agent workflows.
HTML Output Support	Convert complex agent outputs into structured web/UI pages.	Enhances visual navigation and readability.
Hackable Tools	Provide extensible tools and scripts that agents can adapt.	Enables custom, autonomous task workflows.



Issues and Actions

Issue	Action
Agent works in demos	Write 20 real failure cases.
Risky autonomous action	Add step limit + cost cap.
Bad run is unexplained	Add structured traces.
Tempted to fine-tune	Prompt → measure → retrieve.
Tool call is unreliable	Move execution into a harness.
Prompt history is messy	Version + regression test.
Bill is a surprise	Estimate cost per task.
"Model got dumber"	Check data, tools, guardrails.

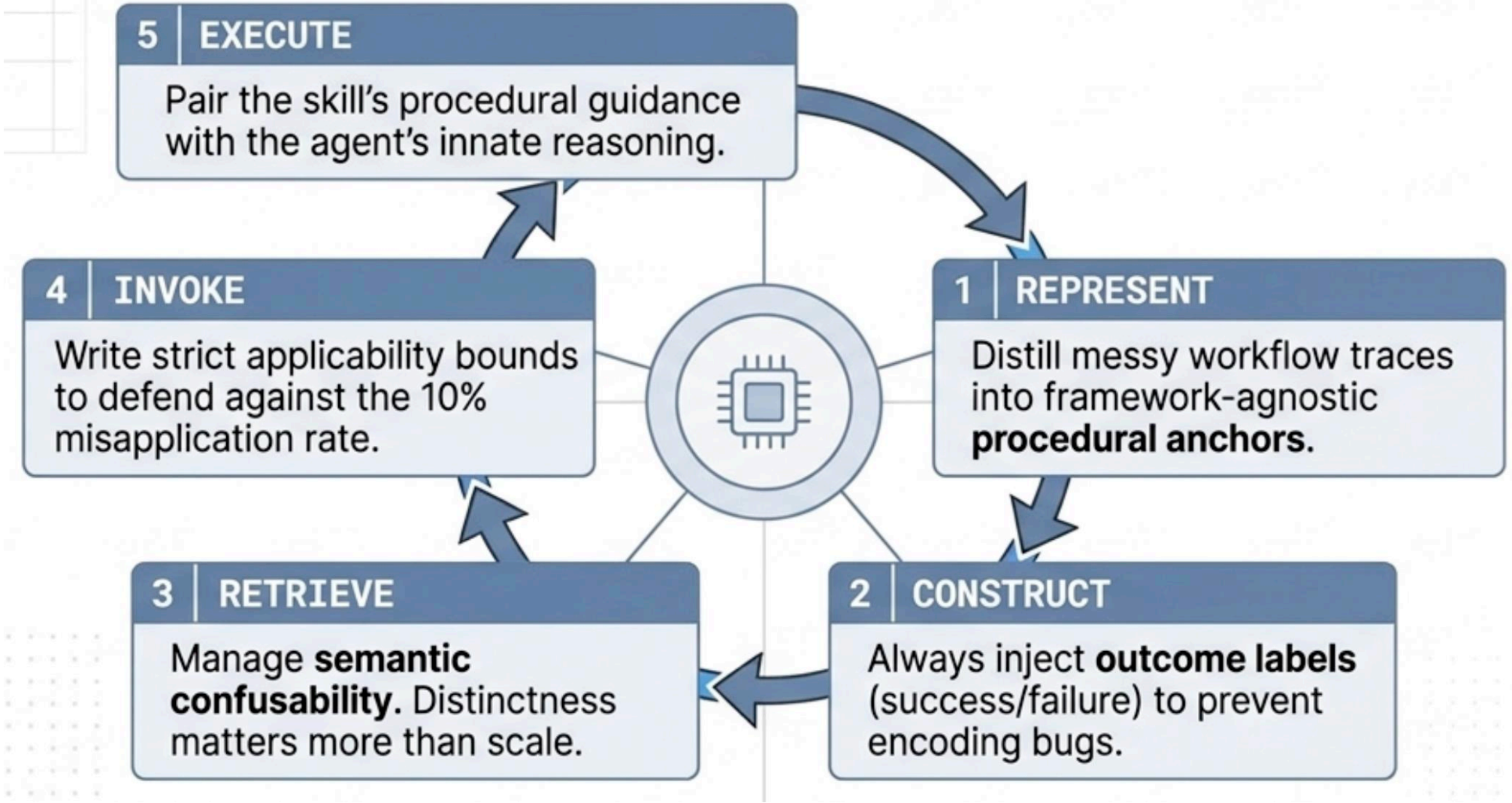


What make a good coder (Human)

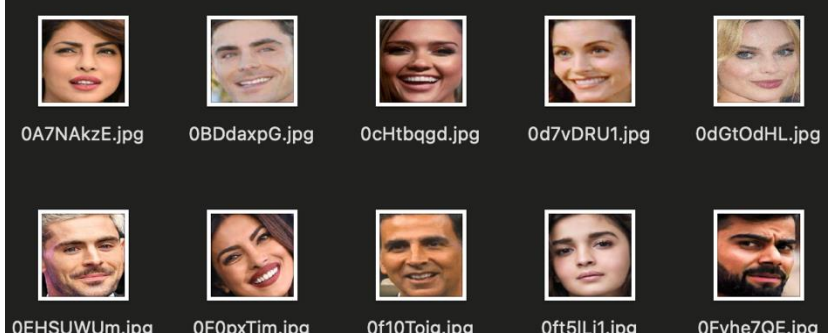
- Two primary individual attributes evaluated as predictors of vibe coding proficiency are Computer Science achievement (r=0.39) and natural language writing skills (r=0.29).
- Educational programs and tool interfaces should continue emphasizing CS fundamentals and technical problem-solving rather than focusing exclusively on prompt engineering



The Skill-Use Lifecycle



Sample Programming Question



- Background
  - You need to cluster the faces in the folder: Faces_Randomized
  - You should start with q1.py, and function c will return the F1 for comparison
- Steps

```
def c(g, p):    return {'Precision': p, 'Recall': r, 'F1': f1}
```

  - Encode the face and cluster the faces using kmeans
  - Save the faces based on their cluster into the json
  - Improve the performance using at least 2 classifiers, such as:
    - check the gender, the blur images,
    - They can be trained with the same set of images
  - Show the improvement with bar plot and conclude your findings.



```
res = {}
for i, l in enumerate(data):
    res[l] = str(labels_random[i])
data_path = ""
# save the result
with open(data_path+"/label_random.json", 'w') as f:
    json.dump(res, f, indent=4)
```

Marking Scheme (50 marks max)

1. 10%: face encoded
2. 10% clustered with k-mean
3. 10% save the faces based on their cluster into a json
4. (5% each, 10% max) Additional e.g.,:
    1. check the gender
    2. check blur
5. 10% Performance
    1. F1 score >0.8 => full mark
    2. formula: min((Best F1 among all json)/8 , 0.1) * 100%



Assignment 3: Face Gender Recognition

- Basic Tasks (50%):
  1. Download 200 face images from online, detect and save the faces to 2 folders (100 male and 100 female)
  2. Load datasets from the folders
  3. Finetune FaceNet as a deep learning-based classifier with the 200 faces
  4. Test with the provided faces in the zip file and save it into json format (As be
  5. Retest the models with different number of freezing layers

```
import json
face_res = {}
face_res['aaa.jpg'] = "M"
face_res['bbb.jpg'] = "F"
# save the json
path_json = '/Users/cpming/Libr
with open(path_json, 'w') as f:
    json.dump(face_res, f)
```

- Advanced Tasks (50%):
  - Repeat the basic tasks without using a pre-train models using CNN
  - Plot the results and conduct an error analysis (find out what and why)
  - Save you best results into json format
- Expected outcomes and notes:
  - Submit a report in pdf to explain your results, the session transcripts to blackboard by 21 Oct (Wed)



face_res.json

```
1  {"aaa.jpg": "M", "bbb.jpg": "F"}
```

Questions?

Email: pakmingcheung@cuhk.edu.hk

